

Learning to Propose, Replaying to Verify: Reusable Neural Guidance for Petri Net Alignments

Alessandro Berti ^{1,*}  and Wil M. P. van der Aalst ^{1,2} 

¹ Process and Data Science (PADS), RWTH Aachen University, Aachen, Germany; wvdaalst@pads.rwth-aachen.de

² Celonis, Munich, Germany

* Correspondence: a.berti@pads.rwth-aachen.de

Featured Application

Interactive conformance checking in which a replay-verified candidate is shown first and exact optimality evidence is added when available.

Abstract

Alignments explain deviations between an event trace and a process model, but difficult traces can make exact computation slow. This paper studies whether one pretrained neural scorer can be reused to propose alignments for different Petri nets and activity vocabularies. LARA (Learned Alignment with Replay Assurance) encodes the net and the complete trace, scores possible moves, and constructs a candidate using a decoder that tracks the marking. Independent replay checks executability and computes the candidate cost. Optional exact search establishes optimality or supplies a replacement. We explain the neural machinery, synthetic training procedure, and verification contract for readers familiar with process mining. On 512 synthetic test inputs, all candidates are legal and 91.0% have optimal cost, compared with 86.9% for the same decoder without learned guidance. The largest improvement occurs when duplicate activity labels require a choice informed by the trace suffix. Experiments on larger synthetic models and two public event logs expose a trade-off between candidate latency and quality. Established approximations generally produce better candidates, and certified mode still runs exact search on every trace. The results support a limited but useful role for reusable learned scoring, with guarantees supplied by process semantics.

Keywords: process mining; conformance checking; alignments; Petri nets; neural networks; foundation models; replay verification

1. Introduction

Conformance checking is useful because a process model and an event log describe different things. The model states which executions are allowed; the log records what was observed. A difference between them may reveal an exception, an incomplete recording, or a model that needs revision. An alignment makes this difference concrete by relating each recorded event to an execution of the model and identifying the extra or missing behavior needed to reconcile the two [1–3]. Its value lies in both a deviation cost and an explanation that can be replayed on the model.

For example, suppose that two transitions have the same activity label, but lead to different branches. The next event alone cannot identify the appropriate transition. Later

Manuscript prepared for: Applied Sciences, special issue “Process Mining: Theory and Applications”.

Copyright: © 2026 by the authors. Submitted to *Appl. Sci.* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

events may reveal which branch gives the best explanation. An exact aligner explores alternatives and returns a minimum-cost explanation. This is an attractive contract, but exploration becomes expensive when many choices interact through concurrency, repetition, or invisible routing. Even when most traces are inexpensive, a few difficult variants can delay a complete analysis. Faster first results could make conformance checking more responsive, provided their meaning is clear.

This paper investigates learned guidance for that first result. A neural network can learn recurring relationships between process structure, observed events, and good alignment choices. Once trained, it can assign scores to moves on a new net-trace pair. The difficulty is that a high score does not prove that a move is enabled, that a complete execution will reach the final marking, or that the resulting cost is minimal. Consequently, replacing formal search with a predictor would change the meaning of the answer unless those obligations were handled explicitly.

LARA, short for *Learned Alignment with Replay Assurance*, assigns these obligations to different components. The network proposes preferences. A decoder follows Petri net firing rules while constructing a candidate. An independent replay routine checks the complete candidate and computes its cost. An optional exact aligner then determines whether that candidate is optimal and replaces it if necessary. The basic principle is simple: a prediction may influence which explanation is tried first, while process semantics determines which claims can accompany it.

The reuse question is central. Training a separate network for every process would create a substantial preparation burden and would weaken the motivation for learning in the first place. We therefore train one scorer on abstract synthetic process structures and reuse its parameters on other inputs. Activity names identify equal labels; the network does not interpret their business meaning. The study follows the idea of pretraining a reusable model, but its scope is narrow: control-flow alignment under fixed unit deviation costs. We use the term *foundation-model approach* in this limited sense and distinguish it from the broad task coverage often associated with foundation models [4,5].

The empirical conclusion is similarly specific. Learned scores improve the same constrained decoder from 86.9% to 91.0% optimal candidates on the synthetic test split, with the strongest gain for duplicate-label choices. All evaluated candidates pass replay. However, exact search is faster on small inputs, and several established approximation methods produce better candidates overall. The present certified mode also runs exact search on every trace; the proportion of candidates that are retained is therefore a quality measure, not a reduction in exact computation. These findings motivate a careful account of where learning contributes and where the decoder or exact backend does most of the work.

The remainder of the paper develops that account in stages. Section 2 introduces the process and learning concepts needed later, then states the research questions and contributions explicitly. Section 3 positions the approach within conformance checking and learned search. Sections 4 and 5 explain inference and training, respectively. Section 6 describes the implementation and interactive workbench. Section 7 evaluates candidate quality, the contribution of learning, and transfer. Sections 8 and 9 discuss the implications and conclude.

2. Preliminaries, Research Questions, and Contributions

The method combines executable process semantics with a statistical predictor. To make the interface between them precise, this section first defines the alignment problem and then introduces the small set of machine learning concepts used by LARA. The em-

phasis is on what each concept permits us to claim. These definitions also establish the research questions and the evidence needed to answer them.

2.1. Traces, Variants, and Labeled System Nets

A trace records the order of activities observed for one case. Let $\mathcal{A}$ be an activity alphabet and let $\sigma = \langle a_1, \dots, a_n \rangle \in \mathcal{A}^*$ be a finite trace. Its length is $|\sigma| = n$; the empty trace is ϵ . An event log $\mathcal{E}$ is a multiset of traces. A *variant* is a distinct activity sequence, and its frequency is the number of cases with that sequence. LARA studies control flow: timestamps, resources, and other event attributes do not enter the scorer or alignment cost.

A labeled system net is $SN = (P, T, F, \lambda, m_0, m_f)$. Here P is a finite set of places, T is a disjoint finite set of transitions, and $F \subseteq (P \times T) \cup (T \times P)$ is the arc relation. We use ordinary arcs with weight one in the formal exposition. The labeling function $\lambda : T \rightarrow \mathcal{A} \cup \{\tau\}$ assigns a visible activity or the invisible label τ . A marking $m : P \rightarrow \mathbb{N}$ assigns tokens to places, with $\mathbb{N}$ including zero. The initial and final markings are m_0 and m_f .

For $t \in T$, its preset $\bullet t = \{p : (p, t) \in F\}$ contains its input places, and its postset $t^\bullet = \{p : (t, p) \in F\}$ contains its output places. The transition is enabled at m when $m(p) \geq 1$ for every $p \in \bullet t$. Firing it produces the marking

$$m'(p) = m(p) - \mathbf{1}[p \in \bullet t] + \mathbf{1}[p \in t^\bullet], \quad p \in P, \quad (1)$$

written $m \xrightarrow{t} m'$. In words, firing consumes a token from every input and produces a token in every output. A transition whose label is τ still changes the marking; it simply does not represent a recorded activity. A *complete firing sequence* starts at m_0 and reaches exactly m_f . We assume that at least one such sequence exists. This assumption ensures that an alignment can always be constructed by using log moves for the trace and model moves for a complete model execution.

Distinct transitions may carry the same label. We therefore distinguish the identifier t from its visible label $\lambda(t)$. Figure 1 illustrates why: t_1 and t_2 both explain an event labeled b , but only t_1 directly leads to c . Matching the label alone does not identify a model execution. Invisible transitions create a related issue: an event may become executable only after unobserved routing steps.

The *complete visible language* $\mathcal{L}(SN)$ contains the label sequences of complete firing sequences after removing all τ labels. Two nets are language equivalent if their complete visible languages coincide. This is a behavioral relation, not a claim that their graphs or internal choices are identical. An *isomorphic renaming* is stronger: it changes identifiers through bijections on places and transitions while preserving arcs, labels, and markings. We use both relations to test whether candidate quality depends on a particular representation.

Process trees provide one source of synthetic nets. A visible leaf represents an activity; sequence executes children in order, exclusive choice (XOR) selects one child, and parallel composition (AND) permits their interleavings. A loop permits repetition. The training generator uses sequence, XOR, and AND; the separate stress generator also includes loops. A fixed compiler converts these operators to Petri net fragments. We call its output *canonical block* only to name this particular compilation, without asserting a universal normal form.

2.2. Alignments: Executability and Deviation Cost

An alignment is needed because a trace may omit a model activity or contain an activity that the model cannot execute at that point. Let $\gg$ denote a skip on one side of an alignment. The permitted moves and their costs are shown in Table 1. A synchronous move consumes one observed event and fires a transition with that label. A log move

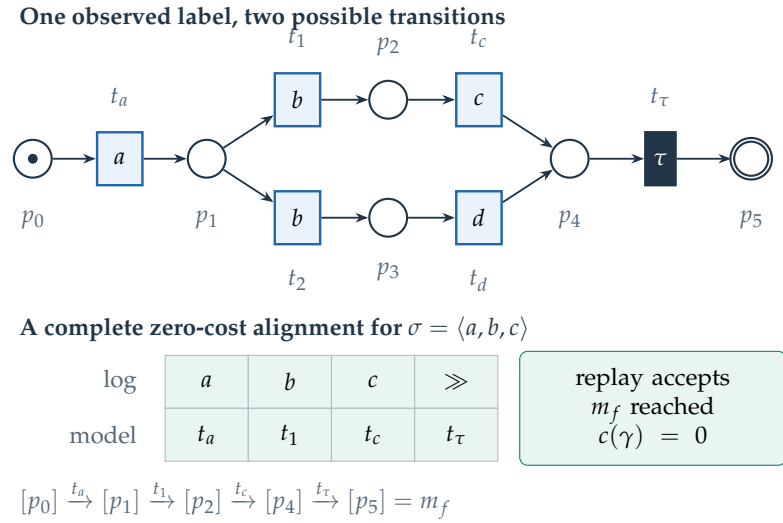


Figure 1. A running example of duplicate-label ambiguity. The token begins at p_0 and must finish at p_5 . Both t_1 and t_2 are labeled b , but their continuations differ. For $\langle a, b, c \rangle$, the upper branch gives a zero-cost alignment. The final invisible transition must still fire even though it consumes no event. Transition identifiers, activity labels, and alignment skips have separate roles.

consumes an event without changing the marking. A model move fires a transition without consuming an event. The pair $(\gg, \gg)$ is excluded because it explains nothing.

For a move sequence γ , its log projection $\pi_L(\gamma)$ removes the model components and skips; its model projection $\pi_T(\gamma)$ removes the log components and skips but retains all transition identities, including invisible transitions. The alignment is *legal* when

$$\pi_L(\gamma) = \sigma \quad \text{and} \quad m_0 \xrightarrow{\pi_T(\gamma)} m_f, \quad (2)$$

and every synchronous move has matching labels. Thus a legal alignment reconstructs the observed sequence exactly and describes a complete executable model path. Merely firing enabled transitions is insufficient: a sequence can stop in the wrong marking.

Let $c(\gamma)$ be the sum of move costs. The optimal alignment cost is

$$\delta(\sigma, SN) = \min_{\gamma \text{ legal for } (\sigma, SN)} c(\gamma). \quad (3)$$

Under these costs, δ is the minimum number of visible insertions and deletions needed to explain the trace by a complete model execution. It does not measure a monetary or business loss. A substitution normally requires one log move and one visible model move, giving cost two. Other cost policies may be appropriate in an application, but the empirical results here concern the policy in Table 1.

An optimum need not be unique. Independent activities can be ordered differently, and invisible routes may differ without changing the visible cost. Consequently, matching

Table 1. Move vocabulary and the fixed costs used in this study. An invisible model move is an actual firing, whereas a skip means that nothing happens on that side.

Move	Form	Meaning	Cost
Synchronous	$(a, t), \lambda(t) = a$	The event agrees with a model firing.	0
Log	$(a, \gg)$	The event has no corresponding firing.	1
Visible model	$(\gg, t), \lambda(t) \neq \tau$	A modeled activity is absent from the trace.	1
Invisible model	$(\gg, t), \lambda(t) = \tau$	Unobserved routing advances the model.	0

the exact teacher's move sequence is a stricter diagnostic than achieving the same cost. We will report both, but use cost equality for optimality. For a replay-verified candidate $\hat{\gamma}$, its *gap* is

$$g = c(\hat{\gamma}) - \delta(\sigma, SN) \geq 0. \quad (4)$$

If exact search does not finish, the gap is unknown. A returned legal candidate still supplies the upper bound $\delta \leq c(\hat{\gamma})$.

2.3. Exact Search, Replay, and Certificates

The shortest-path view explains why finding an alignment and checking one have different computational demands [6]. A search state can be represented as (i, m) , where i events have been consumed and m is the current marking. Synchronous and log moves advance i ; synchronous and model moves update m . Search starts at $(0, m_0)$ and ends at (n, m_f) . Equivalently, one constructs a synchronous product of the process net and a linear net for the trace. The path cost is the alignment cost.

A* prioritizes states by the sum of the cost already incurred and an estimate of the remaining cost. An estimate is *admissible* when it never overestimates the true minimum remaining cost. Marking-equation relaxations obtain useful lower bounds by requiring token balance while relaxing parts of the firing-order constraints [7]. They can guide an exact search because their relationship to the original problem is formally justified. LARA's neural scores have no such property and are never substituted for admissible bounds.

Replay solves a simpler task: given a proposed sequence, check each move in order. It confirms the labels, enabledness, complete trace consumption, and final marking, then sums the stated costs. It does not search for an alternative. The amount of work follows the supplied sequence and its incident arcs rather than the full space of alternative executions. This makes replay a practical check even when finding the best sequence is difficult.

There are therefore three distinct levels of evidence. A raw proposal may fail replay. A proposal that passes replay is an executable explanation with a verified upper bound. Finally, if an exact backend proves that its cost equals δ , it is an optimal explanation. Nonnegative costs also imply that any legal zero-cost alignment is mathematically optimal; the present certification implementation nevertheless calls the exact backend in certified mode. We keep this implementation fact separate from the mathematical observation.

2.4. Learning Terminology for Process Mining Readers

The purpose of the neural component is to reuse patterns from earlier solved alignment problems. In *supervised learning*, a dataset contains inputs and desired outputs. Here the input is a net-trace pair, and the targets come from a replay-checked exact alignment. A neural network is a parameterized numerical function f_θ , where θ collects its adjustable weights. Training changes these weights to reduce a *loss*: a numerical penalty for disagreeing with the targets. After training, *inference* evaluates f_θ with fixed weights on a new input. A saved set of weights and its architecture settings is a *checkpoint*.

An *embedding* is a vector of learned numerical features. A transition embedding can encode aspects of its label, incoming and outgoing structure, and relationship to the observed trace. Its coordinates need not have individually interpretable meanings. Embeddings let the same calculations operate on nets of different sizes: one vector is created per place, transition, or event, and the parameters are shared across them. The number of objects may change without allocating a separate model for each net.

Message passing updates a graph node using information from its neighbors. Typed messages can distinguish a place feeding a transition from a place receiving its output. *Attention* instead lets an object combine information from a collection of other objects using input-dependent weights [8,9]. This is helpful when the next event is ambiguous but a

distant suffix event identifies a likely branch. Attention gives access to context; it does not execute the Petri net or enforce its token rules.

For clarity, a single attention operation can be written as

$$\text{Att}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \quad (5)$$

Rows of Q are *queries*, representing objects seeking information. Rows of K are *keys*, used to score their relevance, and rows of V are the information vectors to combine. The softmax acts row by row: for a vector z , $\text{softmax}(z)_j = e^{z_j} / \sum_k e^{z_k}$. Its entries are nonnegative and sum to one, so each output is a weighted combination of value vectors. The scale $\sqrt{d_k}$ controls the magnitude of the dot products. These arrays are learned linear projections of embeddings, rather than externally supplied queries or keys.

In *self-attention*, queries, keys, and values originate from the same collection, such as all events. In *cross-attention*, they originate from two collections, such as events querying transitions. *Multi-head attention* computes several such combinations using different projections and combines their outputs. A *transformer block* adds a small feed-forward network, residual additions, and layer normalization. Residual additions retain the earlier representation while adding an update; layer normalization rescales a vector's coordinates to stabilize training. A feed-forward network applies learned affine maps and a nonlinear activation to each vector. None of these operations supplies a formal conformance guarantee.

The final output score before any probability conversion is a *logit*. A larger logit means that the network prefers that option under its learned scoring function. Softmax can normalize logits for training, but neither a raw logit nor a normalized score is automatically a calibrated probability that a complete alignment is optimal. A *mask* excludes incompatible options from a calculation. LARA uses exact label equality for one mask and explicit token semantics for enabledness; these are different restrictions.

Training uses three disjoint roles for data. The training split updates θ ; the validation split selects settings or a checkpoint; the test split estimates performance after that selection. A full pass through the training split is an *epoch*. A *batch* groups losses before a parameter update. Gradient-based optimization computes how changing parameters would affect the loss, then adjusts them. *Regularization* discourages brittle solutions; examples here include weight decay, dropout (temporarily suppressing random features during training), and label smoothing (softening a target that would otherwise be fully certain). An *ablation* disables a component while holding the surrounding procedure fixed, so its contribution can be assessed.

2.5. Pretraining, Reuse, and Distribution Shift

Reusable models matter here because a new process should not require another exact-labeled training campaign before candidate generation can begin. *Pretraining* fits the scorer on many synthetic net-trace pairs. *Fine tuning* would subsequently update its weights on a target process. LARA's transfer experiments use the pretrained checkpoint without fine tuning, which we call zero-shot application with respect to that target process. It still receives the target net and trace as inputs.

Foundation models are commonly associated with broad pretraining and adaptation to a range of downstream tasks [4]. Process-specific proposals argue that event data and process representations warrant their own reusable models [5]. LARA explores the reuse principle within one task. Its approximately 1.7 million parameters, small synthetic corpus, fixed costs, and limited training motifs do not demonstrate broad process-mining competence. It is also not a language model: a label such as "approve application" is treated as an equality identifier, without tokenizing or interpreting its words.

Distribution shift occurs when deployment inputs differ from training inputs. Relevant shifts include longer traces, larger nets, new structural motifs, or different label vocabularies. An exact semantics check continues to have the same meaning under such shifts, but learned rankings may become worse. Evaluating reuse therefore requires separate measurements of candidate legality, optimality, and latency. Acceptance by replay demonstrates successful execution of the candidate, not statistical generalization of the scorer.

2.6. Research Questions

The definitions above make it possible to ask questions that distinguish the predictor's value from the guarantees of its wrapper. The study addresses three research questions:

- RQ1. Candidate validity and quality.** How often does candidate generation produce a complete legal alignment, how often is its cost optimal, and how large are its remaining cost gaps? Replay acceptance, exact-cost agreement, and gap distributions provide the evidence in Sections 7.2 and 7.8.
- RQ2. Contribution of learning.** When do learned rankings improve the same decoder compared with deterministic structural choices? A paired ablation and representation-specific analysis isolate effects on duplicate labels and identifier sensitivity in Sections 7.3 and 7.4.
- RQ3. Reuse and practical efficiency.** How do quality and running time change across representations, scales, real event logs, and established approximation methods? Sections 7.4–7.7 assess these changes while keeping the pretrained parameters fixed.

These questions do not assume that a neural candidate is preferable to exact search. A useful answer may identify inputs for which its inference cost buys little or no improvement. Nor does a high retained-candidate rate answer whether certification is faster: that requires measuring a solver that actually uses the candidate to reduce exact work.

2.7. Contributions and Scope

The contributions are organized around an implementable alignment pipeline and evidence about its limits:

- C1. A reusable net–trace scorer.** One graph-and-sequence network predicts transition-aware move preferences for variable-size inputs and previously unseen label vocabularies, without training a network per target net (Section 4.2).
- C2. An explicit verification contract.** The scorer is integrated with marking-aware decoding, independent replay, and exact cost comparison. The resulting output states distinguish a proposal, a verified upper bound, a certified candidate, and an exact replacement (Section 4.5). This contract can also wrap non-neural candidates.
- C3. A controlled synthetic supervision procedure.** Behavior families pair different representations of the same visible behavior, reuse corrupted observations across the pair, and obtain concrete transition targets from exact alignments (Section 5).
- C4. An empirical attribution and transfer study.** The evaluation compares guided and unguided decoding, representation pairs, larger synthetic inputs, existing approximations, and real event logs. It reports where learning helps, where it has no measured effect, and where competing methods perform better (Section 7).

The work concerns completed control-flow traces on labeled system nets. It does not establish guarantees for data-aware or object-centric alignments, online prefix prediction, arbitrary cost policies, or learned lower bounds. The implementation described in Section 6 makes these contributions inspectable through command-line experiments and an interactive workbench.

3. Related Work 282

Several research traditions address the cost of conformance checking, and learning enters this landscape with different possible roles. To locate LARA precisely, this section compares methods by the object they return, the source of their guarantees, and what they reuse across problems. This distinction matters more than whether a method is described broadly as fast, approximate, or intelligent. Table 2 summarizes the main relationships. 283
284
285
286
287

3.1. From Token Replay to Optimal Alignment 288

Conformance checking has long connected event data to executable models. Rozinat and van der Aalst [10] developed replay-based diagnosis of agreement between recorded behavior and a process model. Token replay can identify missing and remaining tokens and is attractive when rapid diagnostics are needed. Later work revisited its implementation and diagnostic behavior to improve scalability [11]. These methods address the practical need for efficient replay, but token deficits and a globally minimum-cost alignment are different output objects. 289
290
291
292
293
294
295

Cost-based alignment expresses deviations through synchronous, log, and model moves [2]. The broader conformance-checking framework treats the resulting correspondence as both a measurement tool and a diagnostic explanation [3]. LARA preserves this transition-aware alignment object. Its verifier does not insert tokens to make a proposed firing possible: it checks whether the supplied model projection really executes from the initial to the final marking. This is why replay in our pipeline is a feasibility check on a candidate, rather than a substitute fitness metric. 296
297
298
299
300
301
302

3.2. Exact Search and Structural Decomposition 303

The synchronous-product formulation turns optimal alignment into a shortest-path problem [6]. Marking-equation and extended-marking-equation methods accelerate search using formally justified estimates of remaining cost [7]. More recent exact systems, including REACH [12], continue to improve search efficiency through algorithmic treatment of explored states and model information. These are important alternatives whenever the requirement is a certified optimum. LARA currently uses an exact backend without modifying its search procedure, so its results should not be interpreted as a comparison against every state-of-the-art exact implementation. 304
305
306
307
308
309
310
311

Complexity results explain why a uniformly inexpensive exact solver is unlikely across broad net classes. The preprint by Schwanen et al. [13] distinguishes the alignment problem on several classes, including hardness results even for structurally restricted models. Such worst-case results motivate practical alternatives, but they do not imply that exact alignment is slow on every real dataset. Our small-input timings make this distinction visible. 312
313
314
315
316
317

Decomposition attacks complexity by splitting the model and observations. Generic Petri net decomposition [14] and single-entry single-exit decomposition [15] define structural conditions under which local analyses are useful. Recomposition then addresses consistency among local results and the relationship between local and global conformance [16]. These approaches have explicit semantic interfaces between fragments. LARA's auxiliary latent regions have no corresponding decomposition theorem and do not generate recomposable local alignments. Although the code includes router and local-head terminology, the evaluated decoder constructs one global candidate from move scores. 318
319
320
321
322
323
324
325

3.3. Approximation by Restricted Search, Compression, and Reuse 326

Approximation methods exploit different sources of regularity. Fixed-horizon alignment limits how far search looks before committing [17]. Discounted alignment changes 327
328

the influence of later deviations through a discounted objective [18]. Sliding-window methods divide long traces into smaller search problems and retain selected boundary candidates [19]. Each reduces some search burden, while a commitment or restricted context can miss a globally better explanation. LARA shares the issue of irrevocable decisions with bounded search, but its ranking uses the complete trace as context.

Compression is another route. Tandem-repeat methods exploit repeated trace segments to reduce the alignment problem before reconstructing a result [20]. Approximation for process trees uses the hierarchical structure of a model to split and compose smaller alignment problems [21]. Neither requires training one neural model per target process. These methods can be strong choices when their structural assumptions match the input.

Subset selection and edit-distance approximation reuse alignments of representative traces to explain other traces [22]. Their reusable object is a collection of solved examples associated with a process, whereas LARA reuses network parameters learned over synthetic nets. The distinction affects preparation costs and failure modes: representative traces must cover relevant behavior, while a pretrained scorer must generalize to the target structure and observation patterns. Our subset comparison uses the concrete configuration in Section 7.6; it does not establish a best possible performance for all subset sizes or representative-selection policies.

Padró and Carmona [23] combine relaxation labeling on a partial-order representation with local optimal search to obtain executable, possibly suboptimal alignments. This is a particularly relevant precedent for separating a correspondence heuristic from completion under process constraints. LARA instead learns graph-and-trace scores across generated instances, then uses bounded marking search and independent replay. Executability checks are valuable for both approaches; they are not a novelty exclusive to neural methods.

3.4. Neural Learning for Event Data and Process Structure

Neural process-mining research provides useful representations but often solves a different task. Recurrent models for predictive monitoring estimate future events, timestamps, or remaining execution time from prefixes [24]. ProcessTransformer applies transformer-based sequence modeling to predictive monitoring [25]. Such predictions do not, by themselves, identify an executable minimum-deviation explanation against a given Petri net. LARA has the complete trace available and returns an alignment whose transition identities can be replayed.

Graph learning brings model structure into the numerical representation. Sommers et al. [26] train a process-discovery technique on synthetic log/model pairs and use graph neural networks to produce Petri nets. This establishes a useful connection to synthetic supervision and reusable structural learning. Its target is a discovered process model, while LARA assumes the model is supplied and learns preferences for explaining a trace against it. Message-passing frameworks [9] and attention [8] supply general computational building blocks; the alignment-specific design lies in input compatibility, transition-aware targets, constrained decoding, and assurance.

The survey and research agenda by Genga and Winter [27] examines artificial intelligence in conformance checking and identifies further opportunities. In this setting, it is useful to distinguish predicting a scalar conformance score, producing a diagnostic object, and proving a property of that object. LARA addresses the second task through learned proposal and constrained construction, while assigning the proof obligations to replay and exact search. It does not infer that successful prediction alone provides conformance assurance.

3.5. Reusable Models and Learning to Guide Search

The foundation-model perspective encourages pretraining a shared representation for reuse. Bommasani et al. [4] discuss this perspective across tasks and modalities, while Rizk et al. [5] argue for models tailored to business-process data. The present study takes a limited step in that direction: one scorer is reused across alignment inputs. Its evidence concerns synthetic motifs and a small number of target logs, so it does not establish a general-purpose foundation model for process mining. This qualification is necessary to align the terminology with the scale and tasks actually evaluated.

More broadly, machine learning for combinatorial optimization studies how learned decisions can complement established solvers [28]. Neural A* [29], for example, couples learned guidance with a differentiable search procedure in path planning. The common motivation is to learn useful preferences over expensive choices. LARA's current integration is simpler: it does not differentiate through A*, alter the exact objective, or establish admissibility of learned values. It learns a first candidate and checks it afterward. A future integration that uses a verified incumbent within exact search would be a different algorithm requiring its own evaluation.

This comparison sets the empirical expectation. LARA should be assessed against exact search, against the same decoder without learning, and against existing approximations. Its contribution cannot be established from a high legality rate alone, because semantics enforces the acceptance condition. The key evidence is whether learned rankings improve useful candidate quality enough to justify their inference cost, and how that evidence changes across representations and datasets.

Table 2. Positioning by output and reuse. Guarantees for a particular approximation depend on its algorithm and configuration; the table does not attribute one common guarantee to the entire family.

Approach	Main object	Reused information	Relation to LARA
Exact alignment	Minimum-cost executable alignment	Model structure and search heuristics	Supplies teacher and certificate.
Decomposition	Local analyses and recomposed results	Structural fragments and their interfaces	Formal fragments differ from latent regions.
Search or compression approximation	Candidate alignment or approximation	Bounded context, repeated structure	Direct candidate-generation competitors.
Subset/edit distance	Transferred trace explanation	Representative solved traces	Reuses examples for a process.
Predictive monitoring	Future event, time, or outcome	Learned event representations	Different target and information horizon.
LARA	Replay-verified candidate; optional exact result	One pretrained net–trace scorer	Learning ranks; semantics checks.

4. Method: From a Net and Trace to a Verified Alignment

The method is designed around a practical division of work. Neural scoring can exploit patterns across solved problems, while explicit firing rules keep the candidate tied to the supplied process model. This section follows one input through representation, scoring, decoding, and verification. At each step, we state both the calculation and why it is needed. Figure 2 gives the overall flow; training is described separately in Section 5 because it changes parameters offline, not during alignment of a target trace.

4.1. Input Representation

The encoder must accept a variable number of places, transitions, and events while preserving the distinctions relevant to alignment. For a pair (SN, σ) , it constructs

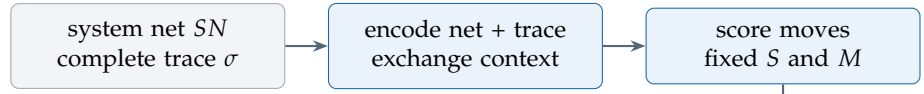
$$X = (X_P, X_T, E, r, z_T, z_\sigma, C). \quad (6)$$

Table 3 summarizes these objects. Place and transition identifiers are sorted to define tensor indices; their strings are not embedded as semantic features. This convention makes the mapping between scores and concrete transitions recoverable during decoding.

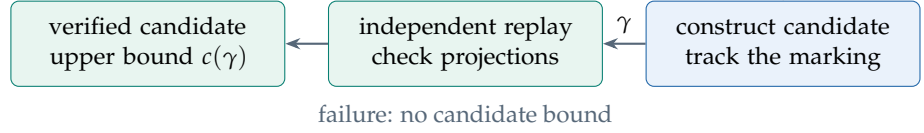
Table 3. Neural inputs. A tensor is a numerical array; its dimensions indicate which process objects index its entries. The teacher alignment is absent from this list.

Object	Shape	Contents and purpose
X_P	$ P \times 5$	Initial/final tokens and place degrees.
X_T	$ T \times 6$	Visibility, transition degrees, costs, and shared input/output context.
E, r	$2 \times 2 F ; 2 F $	Directed edge endpoints and four relation types.
z_T	$ T $	Integer label IDs for transitions.
z_σ	n	Integer label IDs for observed events.
C	$n \times T $	Exact event–transition label compatibility.

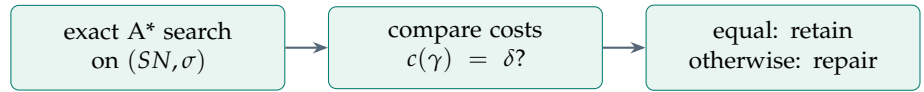
1. Read the problem and score possible moves



2. Construct an execution and check it independently



3. Add optimality evidence when requested



Exact timeout: retain a legal candidate without claiming optimality.

Figure 2. The inference pipeline and its sources of evidence. Blue components produce preferences and a candidate; green components check the candidate or establish an optimum. The upper-bound output is available after replay. Exact search is an additional computation, and a timeout does not turn an unchecked optimum into a certificate. The trained checkpoint is reused across net–trace pairs.

Let $d^-(v)$ and $d^+(v)$ be the incoming and outgoing arc counts of node v . The feature rows are

$$x_p = (m_0(p), m_f(p), d^-(p), d^+(p), d^-(p) + d^+(p)), \quad (7)$$

$$x_t = (\mathbf{1}[\lambda(t) = \tau], d^-(t), d^+(t), c_M(t), c_S(t), s_t), \quad (8)$$

where $c_M(t)$ and $c_S(t)$ are the model and synchronous costs, and $s_t = 1$ when t has a place that is both an input and an output. Initial and final markings tell the network where execution starts and where it should end. Intermediate markings are not input features: they are maintained later by the decoder.

Graph nodes are places followed by transitions. Every original arc is represented in both directions, with four types: place-to-transition consumption, transition-to-place production, reverse consumption, and reverse production. These reverse edges transmit information about a transition’s continuation back toward it. In the running example, the two b transitions have the same immediate input place; information from their different output branches helps distinguish them. The encoder reads the graph directly and does not enumerate reachable markings.

To handle labels outside the training vocabulary, a visible label is converted to an integer by

$$h(a) = 1 + (\text{int}(\text{BLAKE2b}_8(\text{UTF8}(a))) \bmod 8191), \quad h(\tau) = 0. \quad (9)$$

The eight-byte digest is interpreted as an unsigned big-endian integer. The resulting ID selects one of 8192 learned embedding rows. Nearby IDs do not mean similar activities. This bounded representation permits a fixed checkpoint to accept arbitrary label strings, but collisions are possible: two different strings can select the same row.

The compatibility matrix protects the exact label relation:

$$C_{ij} = \mathbf{1}[a_i = \lambda(t_j) \neq \tau]. \quad (10)$$

It is computed from the original labels, not their hashes. A hash collision can affect a preference but cannot authorize a false synchronous match. Conversely, $C_{ij} = 1$ does not imply that t_j is enabled. Label compatibility is checked in the score matrix; token availability is checked in the decoder. Keeping these conditions separate prevents statistical encoding choices from changing the move vocabulary.

The runtime can inspect integer arc weights, but the neural features above do not explicitly encode those weights. Our formal and empirical claims therefore concern the ordinary-net setting used by the study. Passing a richer net object through a software interface would not by itself establish learned generalization to that setting.

4.2. Encoding the Graph and the Complete Trace

The role of the network is to rank plausible choices using context that a local label match cannot see. Its embedding dimension is $d = 96$. Learned affine projections of x_p and x_t initialize graph vectors. A node-type embedding distinguishes places from transitions; transitions also receive their label embedding. In compact notation,

$$h_p^{(0)} = A_P x_p + b_P + e_P, \quad h_t^{(0)} = A_T x_t + b_T + e_T + e_{h(\lambda(t))}. \quad (11)$$

The symbols $A_P, A_T, b_P, b_T, e_P, e_T$ and the label table e are learned parameters. They are shared by all input nets. The model therefore learns transformations of structure and labels rather than a separate vector for each named transition in one process.

Two graph blocks combine global self-attention with typed local messages. For a node v with incoming typed edges $\mathcal{N}_v = \{(u, r) : (u, v, r) \in E\}$, one block computes

$$\mu_v = \frac{\sum_{(u,r) \in \mathcal{N}_v} W_r h_u}{\max(1, |\mathcal{N}_v|)}, \quad (12)$$

$$\tilde{H} = \text{LN}(H + \text{Drop}(\text{MHA}(H, H, H) + \mu)), \quad (13)$$

$$H' = \text{LN}(\tilde{H} + \text{Drop}(\text{FF}(\tilde{H}))). \quad (14)$$

Each W_r is specific to an edge type. MHA is four-head attention based on Equation (5); FF expands the vector to $4d$ coordinates, applies a smooth nonlinear activation (GELU), and projects it back to d . Dropout is active during training only. The average in Equation (12) limits the scale of a message aggregate at high-degree nodes. Self-attention adds access to the whole graph, while typed messages retain an explicit local connection to arc direction and role.

The trace encoder starts with $e_{h(a_i)} + e_i^{\text{pos}}$, adding a learned position vector to each event's label vector. Positions matter because $\langle a, b, c \rangle$ and $\langle c, b, a \rangle$ have different alignment problems despite containing the same labels. A one-layer transformer processes these event vectors with four attention heads. It uses no causal mask: the complete trace, including its suffix, is already available. LARA is therefore an offline alignment method, not a next-event predictor restricted to a running prefix. The position table has 4096 rows; the implementation clamps later positions to its final row. The reported traces are much shorter than this limit.

Two cross-attention operations then connect graph and trace. Let G_T be the graph's transition vectors and H_E the trace vectors. The final event and transition vectors are

$$V = \text{LN}(H_E + \text{MHA}(H_E, G_T, G_T)), \quad (15)$$

$$U = \text{LN}(G_T + \text{MHA}(G_T, H_E, H_E)). \quad (16)$$

The two contexts are calculated from the pre-cross-attention vectors. An event can now reflect which transitions are relevant in this net, and a transition can reflect which events

occur in this trace. The updated vectors are denoted v_i and u_j . These numerical associations explain how the network can use the suffix c to prefer one b transition; they do not guarantee that it learns the appropriate association on every input.

4.3. Move Scores and Their Limits

The network outputs preferences in the same vocabulary used by the decoder. With learned parameters W, w_M, w_L, b_M, b_L , the scores are

$$S_{ij} = \begin{cases} (Wv_i)^\top u_j / \sqrt{d}, & C_{ij} = 1, \\ -10^9, & C_{ij} = 0, \end{cases} \quad (17)$$

$$M_j = w_M^\top u_j + b_M, \quad \ell_i = w_L^\top v_i + b_L. \quad (18)$$

The matrix $S \in \mathbb{R}^{n \times |T|}$ ranks matching transitions for each event. The vector $M \in \mathbb{R}^{|T|}$ ranks model moves. The auxiliary vector $\ell \in \mathbb{R}^n$ is trained to indicate log moves but is not used by the current decoder. The finite value -10^9 effectively excludes incompatible labels from softmax calculations.

A single forward pass calculates these arrays once per net–trace pair. They depend on the complete input, including initial and final markings, but remain fixed as decoding proceeds. In particular, M_j has no event–position index, and neither S nor M is recomputed after a mistaken commitment. This saves repeated neural evaluation but limits adaptation to the decoder’s current state. Scores rank alternatives; move costs remain those in Table 1.

For the running example, the reference checkpoint gives the scores in Table 4. Once t_a has fired, both t_1 and t_2 are enabled and compatible with event b . The score of t_1 is slightly higher, so the decoder takes the branch that can subsequently explain c . The magnitude difference is a preference, not an estimate of a cost saving or a confidence interval. When a unique matching transition is enabled, the synchronous score is not needed at all.

At fixed depth and embedding width, graph self-attention has quadratic dependence on the node count $|P| + |T|$, trace self-attention on n , and cross-attention on $n|T|$. Typed messages additionally traverse the encoded arcs. Avoiding a reachability graph therefore does not make the scorer linear in net size. The parameter count is fixed, but inference cost still grows with the input.

4.4. Marking-Aware Candidate Decoding

The decoder turns preferences into a concrete execution by consulting the current marking at every step. This is necessary because compatible transition labels alone do not enforce the model. Algorithm 1 gives the procedure, parameterized by a catch-up depth d_p and a completion depth d_f . The defaults are $d_p = 8$ and $d_f = 32$; the stress and real-log experiments use $d_f = 64$.

Each breadth-first search explores paths by the number of transition firings. Within an expansion, invisible transitions precede visible transitions, and transitions in the same

Table 4. Reference-checkpoint outputs for Figure 1, rounded to three decimals. Columns follow the sorted transition identifiers. A dash represents the incompatible-label sentinel -10^9 . The unused log scores are $\ell = (-1.682, -1.618, -1.527)$.

Output	t_1	t_2	t_a	t_c	t_d	t_τ
$S_{1,:}$: event a	–	–	.353	–	–	–
$S_{2,:}$: event b	.067	.039	–	–	–	–
$S_{3,:}$: event c	–	–	–	.876	–	–
M	–1.357	–1.401	–1.030	–1.830	–1.837	2.519

Algorithm 1 Construct a candidate with bounded model-move searches.

Require: $SN, \sigma = \langle a_1, \dots, a_n \rangle$, fixed scores S, M , depths d_p, d_f

```

1:  $m \leftarrow m_0; \gamma \leftarrow \langle \rangle$ 
2: for  $i = 1, \dots, n$  do
3:    $B \leftarrow \{t \in T : t \text{ enabled at } m, \lambda(t) = a_i\}$ 
4:   if  $B = \emptyset$  then
5:     BFS from  $m$ , depth at most  $d_p$ , to a marking enabling label  $a_i$ 
6:     if a path is found then
7:       Append its model moves to  $\gamma$  and fire them on  $m$ 
8:       Recompute  $B$  at  $m$ 
9:     end if
10:  end if
11:  if  $B \neq \emptyset$  then
12:     $t_j \leftarrow \arg \max_{t_j \in B} S_{ij}$  ▷ unique match needs no ranking
13:    Append  $(a_i, t_j)$  to  $\gamma$ ; fire  $t_j$  on  $m$ 
14:  else
15:    Append  $(a_i, \gg)$  to  $\gamma$  ▷ marking unchanged
16:  end if
17: end for
18: BFS from  $m$ , depth at most  $d_f$ , to  $m_f$ 
19: Append and fire the completion path if found (empty if already at  $m_f$ )
20: return  $\gamma$  for independent replay, including when completion failed

```

group are ordered by decreasing M_j . The search keeps a visited-marking set and stops at the first goal marking found. It does not minimize deviation cost: a short path containing a visible move may be considered before a longer path containing only zero-cost invisible moves. Depth limits bound path length but do not impose a fixed limit on the number of explored markings; branching can still make a bounded search expensive.

If no bounded path enables the event, the decoder makes a log move. If a matching transition is already enabled, it synchronizes immediately. This policy never explicitly compares the cost of skipping that event with the downstream consequences of synchronizing it. It also never revises earlier choices. The simplicity gives a low-overhead baseline, but it explains the failure pattern examined in Section 7.8.

In unguided mode, all learned rankings are removed and the forward pass is skipped. The same firing rules, depth limits, and invisible-first policy remain. Ties among enabled synchronous transitions follow ascending transition-name order; ties in model search follow the runtime's reverse name order. The difference between guided and unguided modes therefore concerns ranking, not relaxed legality. Identifier order is deterministic but has no business meaning, which motivates the renaming experiment.

4.5. Replay Verification and Exact Certification

Verification is needed even though the decoder fires enabled transitions: a bounded completion can fail, and independent checking protects the interface against malformed candidates. Replay reconstructs the marking sequence from m_0 , validates each transition identity and synchronous label, checks that the log projection is exactly σ , and requires the final marking to equal m_f . Only then is the summed cost advertised as an upper bound. A failed proposal can be retained for diagnostics, but it is not a verified alignment result.

The assurance contract follows immediately from the alignment definition.

Proposition 1 (Verified candidate and cost equality). *If replay accepts $\hat{\gamma}$, then $\delta(\sigma, SN) \leq c(\hat{\gamma})$. If an exact aligner establishes $\delta(\sigma, SN)$ and $c(\hat{\gamma}) = \delta(\sigma, SN)$, then $\hat{\gamma}$ is optimal, irrespective of whether its move sequence equals the exact aligner's witness.*

Proof. Replay acceptance establishes that $\hat{\gamma}$ belongs to the feasible set in Equation (3). Its cost is therefore at least the minimum over that set. Equality means that it attains the minimum. No property of the neural scores is needed. $\square$

In FAST mode the system returns the candidate, replay status, and verified cost when available. In CERTIFIED mode it additionally calls PM4Py’s state-equation A* aligner. If an optimal exact result is available, it retains a legal equal-cost candidate; otherwise it returns the exact alignment as a replacement, with replay checks on that result. If exact search times out or fails, a legal candidate remains an upper bound and optimality stays unconfirmed. Table 5 lists the externally meaningful outcomes.

The current exact backend starts its own search. It does not consume the candidate as an incumbent solution, prune states using its cost, or use learned scores in its heuristic. Consequently, a candidate that is already optimal still incurs exact work in certified mode. The wrapper separates latency to a usable candidate from latency to a certificate; it does not yet accelerate the certificate itself. These same steps can be applied to a candidate produced by a classical approximation.

Table 5. Meaning of result states. Certification is conditional on successful exact computation, not on a mode name or a learned score.

State	Evidence obtained	Claim available
Rejected proposal	Replay fails or no proposal is produced	No candidate upper bound.
Verified candidate	Replay succeeds	Executable explanation; $\delta \leq c(\hat{\gamma})$.
Certified candidate	Replay succeeds; exact optimum has equal cost	Candidate attains the minimum cost.
Exact replacement	Exact optimum available; candidate rejected or has larger cost	Returned exact alignment is optimal.
Exact timeout	Exact optimum remains unavailable	Only the candidate’s existing replay evidence, if any.

5. Data Generation and Training

Training a reusable scorer requires examples that separate process behavior from the way a net happens to represent it. It also requires trustworthy targets: synthetic corruption tells us how an observation was produced, but not necessarily its cheapest explanation. This section describes the complete path from a generated behavior family to input tensors, exact alignment targets, and parameter updates. All generation and loss details needed to understand the reference model are integrated here.

5.1. Why Generate Behavior Families?

An isolated pair of a random net and a noisy trace would teach the network about one representation at a time. It would give much less control over whether a performance change comes from different behavior or merely different routing. A *behavior family* addresses this by grouping two nets with the same complete visible language and two shared observations:

$$\mathcal{B} = (SN_1, SN_2, \sigma_1, \sigma_2), \quad \mathcal{L}(SN_1) = \mathcal{L}(SN_2). \quad (19)$$

Each family expands into four supervised inputs, (SN_r, σ_k) for $r, k \in \{1, 2\}$. Both representations see exactly the same observations. Figure 3 shows this expansion and the checks that precede writing the rows.

The entire family belongs to one split before expansion. This avoids placing two views of the same generated family in training and test. It does not make all visible languages, motifs, or net topologies disjoint across splits: independently generated families can repeat the controlled structures. The test split measures new family instances within this generator, not transfer to wholly unseen motif classes.

Before generating nets, a deterministic quota assigns the requested number of families to four equally weighted motifs. Minimum family counts are reserved per motif, remaining slots are distributed by the weights, and the plan is shuffled with a split-specific seed. If a generation attempt fails, another family index is tried for the same planned motif. This prevents rejection from silently shifting the corpus toward easier model classes. The global reference seed is 13; separate seeds derived from the split and family index control structure, representation, and corruption.

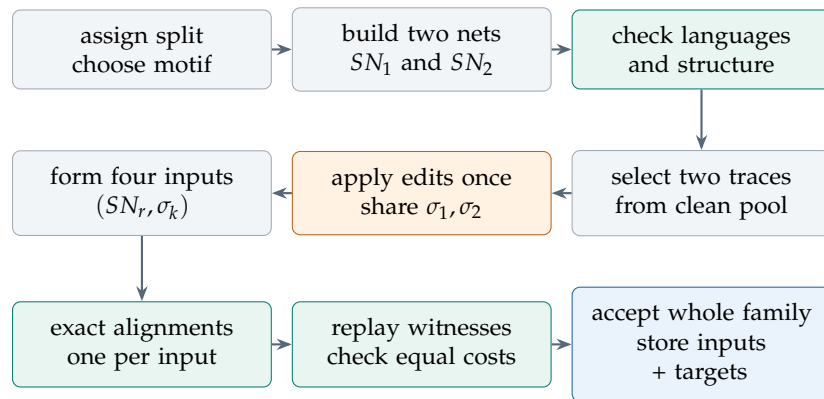
5.2. Four Motifs and Their Representation Pairs

The motifs target distinct reasons why a label sequence may be difficult to align. Table 6 specifies their visible core languages. The three controlled motifs append two fresh activities shared by both representations. If their labels are u, v , the complete language is $\{wuv : w \in \mathcal{L}_{\text{core}}\}$. Fresh labels prevent the suffix from introducing another core ambiguity. The implementation selects the first unused activity names: the duplicate/silent and M motifs append D, E , while the concurrency motif appends C, D .

5.2.1. Ordinary Trees and Identifier Renaming

This pair tests whether a candidate depends on incidental naming. A random process tree has 4–12 visible leaf occurrences and depth at most six. Sequence, XOR, and AND have selection probabilities 0.4, 0.3, and 0.3. The alphabet size is approximately 70% of the visible leaf count, so different leaves can carry the same label. The compiler maps leaves to visible transitions, sequences to linked fragments, XOR branches to shared entry and exit places, and AND to invisible split/join routing.

The second net is an isomorphic copy with renamed place and transition identifiers. Labels, arcs, markings, and weights remain unchanged under the corresponding bijections. Its firing possibilities and optimal costs are therefore unchanged. Sorting the new identifiers



If a required check fails, replace the attempted family with another of the same planned motif. Corruption history never becomes a neural input.

Figure 3. Data generation as a sequence of explicit checks. One family supplies two behaviorally equivalent representations and two observations shared across them. Each of the four rows receives its own exact witness. A family is written only after all rows pass, keeping both the split assignment and the planned motif balance intact.

Table 6. Motif design. The listed languages omit the shared suffix for the three controlled motifs. “Canonical” identifies the generator’s fixed block compiler.

Motif	Core behavior	Representation pair
Ordinary tree	Sampled tree language	Canonical block net and isomorphic identifier renaming.
Duplicate/silent	AB, AC	Two A transitions that select a branch; one A followed by invisible routing.
Concurrency	AB, BA	True parallel execution; explicit choice between the two sequences.
M pattern	B, AC, CA	XOR of B and parallel A, C ; non-free-choice M net.

can, however, change an unguided tie-break. This pair distinguishes an improvement in handling identifiers from an improvement on a new process behavior.

5.2.2. Duplicate Prefix and Silent Routing

This pair isolates a choice that may have to be made before its distinguishing event arrives. In the duplicate-prefix net, two enabled transitions are both labeled A . One leads to B and the other to C . Firing A therefore commits to a branch. In the silent-routing net, there is one shared A transition followed by two invisible alternatives that enable B or C . Both nets generate AB or AC before the common suffix, but the visible ambiguity occurs at a different point. Figure 4 draws both cores.

For an observation beginning with A, C , selecting the appropriate duplicate A requires looking beyond the current event. The silent-routing representation can postpone its branch selection until trying to enable C . If guidance helps mainly on the first net, that is evidence about early ambiguity, rather than a universal advantage over the same decoder.

5.2.3. Parallelism and Explicit Interleaving

This pair tests a change in the representation of concurrency. One net uses an invisible split to enable A and B in parallel, followed by a join. The other uses an XOR between the sequences AB and BA . Their complete visible traces coincide, but their causal structures

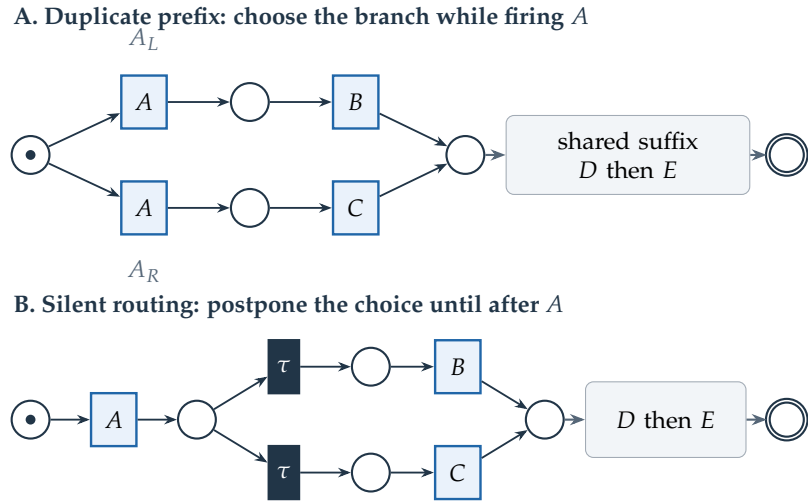


Figure 4. Equal visible languages with different decision points. Top: choosing a concrete A transition commits to B or C . Bottom: a shared A precedes invisible branch routing. The appended suffix D, E is identical in both nets. The visible language is $\{ABDE, ACDE\}$, although transition identities and silent moves differ.

differ: simultaneous availability in one representation becomes a choice between sequential executions in the other. The equivalence is therefore trace-language equivalence, not equivalence of partial-order behavior. This distinction matters when interpreting representation agreement as evidence of transfer.

5.2.4. Block Structure and the Non-Free-Choice M Pattern

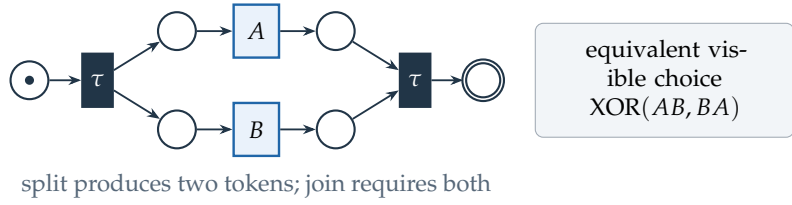
This pair introduces a conflict that cannot be described merely as a shared choice among transitions with identical input places. The block representation is $XOR(B, AND(A, C))$. In the M representation, an invisible split places one token in each of two places. A consumes the first, C consumes the second, and B consumes both. Therefore B conflicts with both A and C , while A and C can occur in either order. The presets of conflicting transitions are unequal, giving the non-free-choice structure. Appropriate output places and a join complete the A, C branch. Figure 5 shows the token-level construction together with the parallelism pair.

5.3. Checking Behavioral Equivalence and Usable Structure

The representation checks protect the interpretation of a family. If the two nets admitted different visible traces, a cost difference could be semantically correct rather than a representation effect. The generator enumerates complete visible traces and records whether enumeration finishes. Default limits are 5000 explored states, 10,000 complete traces, and visible length 20, with an additional firing bound to prevent unbounded invisible exploration. A comparison is labeled *exact* only if neither enumeration is truncated. A matching bounded prefix is not treated as proof of full language equality.

Generation rejects mismatching languages, an empty complete language, an unreachable final marking, or a dead transition. Here dead means that the structural exploration finds no reachable firing of that transition. These checks are useful filters, but should not be read as a general proof of every workflow-net soundness property for arbitrary inputs. The default training configuration also rejects families whose equivalence check is only bounded. All accepted rows in the reference training, validation, and test corpus have exact language certificates within the configured bounds.

A. Parallel execution and explicit interleavings



B. The M pattern and its block representation

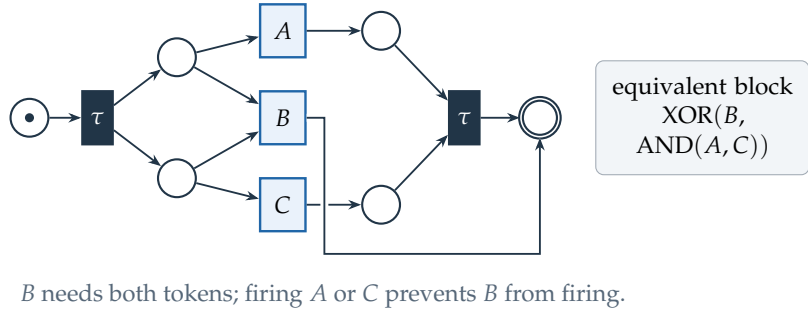


Figure 5. Two additional changes of representation. The parallel core and the explicit interleaving expression both admit AB and BA . In the M core, B consumes both central tokens, while A and C consume one each and later join; its visible language is $\{B, AC, CA\}$. The right-hand expressions denote the corresponding block structures. Shared fresh suffixes are omitted.

The cost comparison used later has a formal rationale. Let $d_{ID}(\sigma, w)$ be insertion/deletion distance, with unit cost for each inserted or deleted visible activity. With the costs in Table 1,

$$\delta(\sigma, SN) = \min_{w \in \mathcal{L}(SN)} d_{ID}(\sigma, w). \quad (20)$$

Every legal alignment selects a complete visible model word and a corresponding sequence of insertions, deletions, and matches. Conversely, such an edit sequence can be combined with a firing sequence for that word, inserting its zero-cost invisible moves. Thus language-equivalent nets have equal optimal visible-deviation costs, even though their optimal transition sequences may differ. This equality would require re-examination if costs depended on transition identity or charged invisible routing.

5.4. Producing Observations by Trace Corruption

Corruption creates deviations without changing the reference process. For each accepted family, the enumerated language is sorted. If it contains more than 16 words, the family random generator selects 16 indices and restores them to language order. Otherwise the whole language is retained. The first two entries of this pool supply clean traces, wrapping around when only one entry is available. These choices and their corruption are deterministic given the configured seeds.

Each observation is kept clean with probability 0.2. Otherwise it receives one, two, or three edits with probabilities 0.5, 0.3, and 0.2. Table 7 lists the operation weights. Operations are drawn again after each edit and act on the current trace. Inapplicable operations, such as a swap on a one-event trace, fall back to insertion of the outside label `__00D__`. Traces are capped at length 128. The realized frequencies can therefore differ from the nominal weights, especially for short observations.

The same corrupted observation is then paired with both nets. This is essential to the comparison: applying different noise to the two representations would confound behavior with observation quality. The corruption history is metadata, not a neural input and not

Table 7. Trace-corruption operations. Examples start from *ABDE* and illustrate individual edits; multi-edit traces apply several operations in sequence. Weights are conditional on drawing an edit operation.

Operation	Weight	Effect	Example
Delete	.15	Remove an event.	<i>ADE</i>
Insert	.15	Insert a label from the family alphabet.	<i>ABBDE</i>
Substitute	.15	Replace an event label.	<i>ACDE</i>
Swap	.15	Exchange neighboring events.	<i>BADE</i>
Repeat	.15	Duplicate one event.	<i>ABDEE</i>
Prefix truncation	.10	Remove a beginning segment.	<i>DE</i>
Suffix truncation	.10	Remove an ending segment.	<i>AB</i>
Outside insertion	.05	Insert the special outside label <i>X</i> .	<i>ABXDE</i>

the alignment target. An edit can produce another fitting trace; several edits can cancel or admit an explanation cheaper than undoing them. For example, substituting *C* for *B* in *ABDE* produces *ACDE*, which still fits the duplicate/silent family. Exact alignment is therefore performed after corruption.

5.5. Exact Supervision and Family-Level Acceptance

Each expanded row is sent to the PM4Py state-equation A* teacher with transition-aware output. The reference generation imposes no per-trace timeout. Results are cached within a split using a signature of the net, the observed sequence, and the cost-model version. Caching reduces repeated teacher work without changing the targets. A different representation still requires its own concrete transition witness.

Every exact witness is replayed independently. The initializer requires an optimal result, checks all moves and both projections, and verifies that the paired nets give equal optimal costs for the same observation when their language equivalence was exact. Rows are buffered until the entire family passes; a failure replaces the family rather than writing only its successful rows. For the reference corpus, the manifest records 25 generation-time `ValueError` attempts that were replaced. It records no exact-alignment, replay, or paired-cost rejection. These are observations about this run, not a guarantee that future configurations cannot fail those gates.

From one optimal witness γ^* , the code derives three move targets. The synchronous target y_i is the zero-based index of the transition used for event i , or -1 when there is no active synchronous target. The binary target q_i is one exactly when the event is a log move. The binary target b_j is one when transition t_j occurs at least once as a model move anywhere in γ^* . Invisible model moves are included in b . This last target does not record the position or multiplicity of model moves, matching the deliberately coarse vector M .

Teacher identity is not unique when several alignments are optimal. The generator uses one particular witness. For the duplicate-prefix motif, it further masks all synchronous targets when the observation contains both distinguishing labels *B, C* or neither. This is the implemented heuristic for avoiding ambiguous identity supervision, not a general theorem characterizing all unique optima. Log, model, and cost targets remain active, and the witness itself remains in the stored row.

5.6. A Complete Worked Training Row

A concrete row helps separate input information from the answer supplied during training. Consider stored sample `train-000048`. Starting from *ABDE*, the generator swaps the final events and substitutes *C* for *B*, yielding $\sigma = \langle A, C, E, D \rangle$. Its duplicate-prefix net

is the upper representation of Figure 4, followed by the suffix. Write A_L, A_R for its two A transitions, with A_L leading to B and A_R to C .

The place order is $(p_D, p_E, p_0, p_f, p_L, p_R)$ and the transition order is (D, E, A_L, A_R, B, C) . The core ends at p_f , followed by D to p_D and E to p_E ; the final marking is $[p_E]$. In these orders, the numerical feature matrices are

$$X_P = \begin{bmatrix} 0 & 0 & 1 & 1 & 2 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 2 & 2 \\ 0 & 0 & 2 & 1 & 3 \\ 0 & 0 & 1 & 1 & 2 \\ 0 & 0 & 1 & 1 & 2 \end{bmatrix}, \quad X_T = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}. \quad (21)$$

Every transition has the same local feature row, yet the graph connections and labels differ. This demonstrates why local degrees alone cannot resolve the duplicate choice.

The graph uses place indices $0, \dots, 5$ and transition-node indices $6, \dots, 11$. Table 8 specifies all edges compactly. For each triple (p, t, q) , insert edges $(p, t), (t, p), (t, q), (q, t)$ with types $(0, 2, 1, 3)$. Thus E has 24 columns and r repeats this four-type sequence six times. This explicitly recovers the arc information that distinguishes the otherwise equal rows of X_T .

The label IDs and exact compatibility matrix are

$$z_T = (2587, 4475, 8075, 8075, 6408, 5878), \\ z_\sigma = (8075, 5878, 4475, 2587), \quad (22)$$

$$C = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}. \quad (23)$$

The first event can match either A transition. The third event is compatible with E , even though E is not enabled initially. The separate position indices $(0, 1, 2, 3)$ are added by the trace encoder. Equations (21)–(23) and Table 8 specify every component of X ; no clean trace or exact cost is among these inputs.

The stored teacher witness is

$$\gamma^* = \langle (A, A_R), (C, C), (E, \gg), (D, D), (\gg, E) \rangle, \quad \delta = 2. \quad (24)$$

Table 8. Full edge specification for the worked input. The readable transition name identifies a row; the three integers are graph-node indices in Equation (21).

Transition	Input place p	Transition node t	Output place q
D	3	6	0
E	0	7	1
A_L	2	8	4
A_R	2	9	5
B	4	10	3
C	5	11	3

It takes the branch consistent with C , skips the early E , synchronizes D , and finishes with a model move E . Another optimum could insert D before synchronizing E and then skip the final observed D . The chosen witness gives

$$y = (3, 5, -1, 0), \quad q = (0, 0, 1, 0), \quad b = (0, 1, 0, 0, 0, 0). \quad (25)$$

These arrays and δ are the supervision. In particular, target $y_1 = 3$ names A_R in the stored zero-based transition order; it is not activity ID 3. This distinction is important when implementing transition-aware learning.

5.7. Training-Time Relabeling and Corpus Composition

Label relabeling discourages the scorer from solving controlled examples by memorizing a small set of synthetic IDs. On every training visit, probability 0.5 triggers a consistent random remapping of distinct nonzero IDs in the net and trace to distinct values in $\{1, \dots, 8191\}$. The graph, exact compatibility mask, and targets remain unchanged. For example, mapping the IDs for A, B, C, D, E to 101, 205, 309, 413, 517 would give $z'_T = (413, 517, 101, 101, 205, 309)$ and $z'_\sigma = (101, 309, 517, 413)$. This example illustrates an augmentation, not a recorded random draw.

ID zero stays reserved for invisible transitions, and existing hash collisions remain collisions. Because embeddings differ across IDs, this augmentation encourages robustness but does not prove invariance under every label renaming. Validation and test use their original hashed IDs, and dropout is disabled there. The reference split sizes appear in Table 9. The test traces average 3.6 events and their exact costs range from zero to seven, so this is a small controlled corpus rather than a comprehensive sample of industrial process complexity.

5.8. Learning the Move Preferences

The loss expresses which aspects of an exact witness the network should imitate. It does not differentiate through Petri net execution or prove the predicted sequence correct. Let $I = \{i : y_i \geq 0\}$ and $J_i = \{j : C_{ij} = 1\}$. When y_i is used as a matrix column below, its stored zero-based index is converted to the corresponding column. For $i \in I$, normalize only compatible synchronous scores:

$$p_{ij} = \frac{e^{S_{ij}}}{\sum_{k \in J_i} e^{S_{ik}}}, \quad j \in J_i. \quad (26)$$

Cross-entropy penalizes putting little mass on the teacher transition. With smoothing $\epsilon_s = 0.05$, it is

$$\mathcal{L}_{\text{sync}} = -\frac{1}{|I|} \sum_{i \in I} \left[(1 - \epsilon_s) \log p_{i, y_i} + \frac{\epsilon_s}{|J_i|} \sum_{j \in J_i} \log p_{ij} \right]. \quad (27)$$

This assigns 95% of the target mass to the teacher and spreads the remaining 5% uniformly across compatible transitions. In the worked row, the two A transitions receive target masses .025 and .975. An event with a single compatible transition contributes zero syn-

Table 9. Reference corpus with seed 13. A family contributes four rows and remains inside one split. All four motifs are balanced within each split.

Split	Families	Net-trace rows	Families per motif
Training	512	2048	128
Validation	128	512	32
Test	128	512	32

chronous loss, because there is no label-compatible choice to learn. If I is empty, the term is zero.

For binary decisions, define $\text{BCE}(z, u) = \log(1 + e^z) - uz$, evaluated with a numerically stable implementation. A positive target encourages a high score and a negative target a low score. Binary smoothing maps target zero to .03 and target one to .97. The other move losses are

$$\mathcal{L}_{\log} = \frac{1}{n} \sum_i \text{BCE}(\ell_i, .94q_i + .03), \quad \mathcal{L}_{\text{model}} = \frac{1}{|T|} \sum_j \text{BCE}(M_j, .94b_j + .03). \quad (28)$$

An empty output contributes zero. The log head receives supervision even though decoding does not yet consult it. This mismatch between training and use is one reason to examine the decoder separately from the network.

5.9. Auxiliary Cost and Region Objectives

The reference architecture also contains auxiliary region and cost heads. Their role must be explained because they contribute to the training objective, although they do not produce local executable alignments or participate in certification. A learned router maps each transition and event vector to a softmax distribution over $R = 6$ latent regions, denoted Q_{jr}^T and Q_{ir}^E . “Region” means an internal grouping in representation space; it is not a verified subnet decomposition.

For each region, normalized weighted means summarize transitions and events:

$$\bar{u}_r = \frac{\sum_j Q_{jr}^T u_j}{\max(10^{-6}, \sum_j Q_{jr}^T)}, \quad \bar{v}_r = \frac{\sum_i Q_{ir}^E v_i}{\max(10^{-6}, \sum_i Q_{ir}^E)}. \quad (29)$$

A small feed-forward network transforms their concatenation into h_r . Two heads produce $\hat{l}_r = \text{softplus}(w_l^\top h_r + b_l)$ and $\hat{u}_r = \hat{l}_r + \text{softplus}(w_u^\top h_r + b_u)$, where $\text{softplus}(x) = \log(1 + e^x)$. This enforces nonnegative ordered endpoints. It does not enforce that the true alignment cost lies between them. Only their aggregate upper endpoint is supervised:

$$\hat{c} = \sum_r \hat{u}_r, \quad \mathcal{L}_{\text{cost}} = H_1(\hat{c} - \delta), \quad H_1(e) = \begin{cases} e^2/2, & |e| < 1, \\ |e| - 1/2, & |e| \geq 1. \end{cases} \quad (30)$$

The smooth-L1 penalty is quadratic near the target and grows linearly for larger errors. The code names these endpoints “lower bounds” and “upper bounds”, but they are predictions without admissibility guarantees. We therefore never use those names as formal claims. Sketch and uncertainty heads are also exposed by the architecture; they have no direct supervised loss and do not enter the decoder.

Three regularizers influence the latent grouping. Let U_p be the set of distinct transitions incident to place p , ignoring arc direction. The boundary term is

$$\mathcal{L}_{\text{boundary}} = \frac{\sum_{p \in P} \sum_{\{j,k\} \subseteq U_p} (1 - \sum_r Q_{jr}^T Q_{kr}^T)}{\sum_{p \in P} \binom{|U_p|}{2}}, \quad (31)$$

set to zero when the denominator is zero. It favors similar assignments for transitions sharing a place. A pair sharing more than one place is counted for each shared place. Define the mean load $a_r = (\sum_j Q_{jr}^T + \sum_i Q_{ir}^E) / (|T| + n)$ and entropy $H(q) = -\sum_r q_r \log q_r$. Then

$$\mathcal{L}_{\text{balance}} = \frac{1}{R} \sum_r (a_r - 1/R)^2, \quad (32)$$

$$\mathcal{L}_{\text{entropy}} = \frac{1}{2} \left(\frac{1}{|T|} \sum_j H(Q_j^T) + \frac{1}{n} \sum_i H(Q_i^E) \right). \quad (33)$$

For an empty trace, these terms use transitions only; logarithm arguments are clamped at 10^{-8} in code. Minimizing the positive entropy term encourages concentrated assignments, while balance discourages using a single region for everything. These competing preferences shape representations, not verified process fragments. No ablation in this study isolates the benefit of these auxiliary terms.

5.10. Optimization and Checkpoint Selection

The full objective, with the weights used by the reference run, is

$$\begin{aligned} \mathcal{L} = & \mathcal{L}_{\text{sync}} + \mathcal{L}_{\text{log}} + \mathcal{L}_{\text{model}} + .03\mathcal{L}_{\text{cost}} \\ & + .01\mathcal{L}_{\text{boundary}} + .01\mathcal{L}_{\text{balance}} + .001\mathcal{L}_{\text{entropy}}. \end{aligned} \quad (34)$$

Each component is first reduced within one input pair; the resulting losses are averaged over 16 pairs per optimizer step. The implementation processes variable-size pairs separately and accumulates gradients. AdamW [30] updates the parameters with initial learning rate 5×10^{-4} and weight decay 3×10^{-3} . Gradient norms are clipped at one. Dropout probability is .30.

Validation loss controls learning-rate reduction by factor .5 after two plateau epochs, with a minimum rate of 10^{-5} . Early stopping has patience six and requires an improvement of at least 5×10^{-4} . The reference run executes 50 epochs and selects epoch 49. The mean epoch duration is approximately 41 seconds on the reported CPU setup. Figure 6 shows the full recorded loss curves. Lower validation than training loss is plausible because validation disables dropout and random label remapping; the two losses are evaluated under different perturbation conditions.

Replay legality and exact optimality are evaluated outside this differentiable objective. The network learns to imitate aspects of exact solutions; the semantic checks remain explicit computations at use time. This is the reason a poor prediction may reduce candidate quality without invalidating the meaning of a successful replay or an exact certificate.

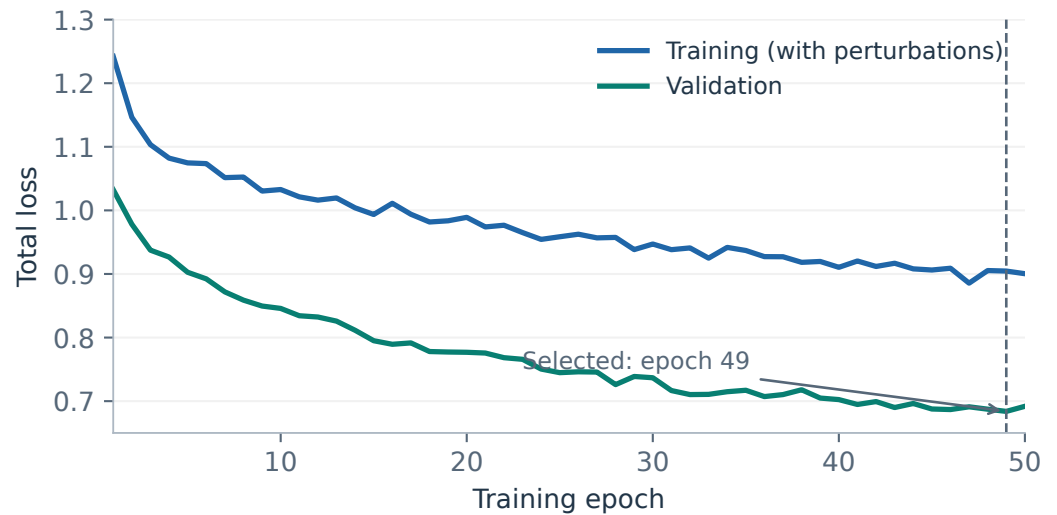


Figure 6. Recorded total training and validation losses. The dashed line identifies the checkpoint selected at epoch 49. These are the weighted objectives in Equation (34), not alignment costs or certificate rates. Training includes dropout and random label remapping; validation does not.

6. Implementation and Interactive Use

An alignment method becomes useful to an analyst only when its explanations and evidence can be inspected. This section describes how the mathematical stages are implemented, how the workbench exposes their different outcomes, and how experiments can be reproduced. The interface treats a candidate and its later exact comparison as separate results so that a fast first explanation is never silently presented as an optimum.

6.1. Software Components and Interfaces

The implementation is available at <https://github.com/fit-alessandro-berti/lara-align>. It uses PyTorch for neural computation and PM4Py for Petri net and event-log objects, process discovery, and exact alignment [31]. The package provides Python and command-line interfaces; the workbench uses Streamlit. Table 10 maps the conceptual stages to their implementation modules. This separation allows candidate-generation experiments to reuse the same independent verifier.

An alignment move stores the observed label, transition name, and transition label where applicable. This avoids ambiguity when several transitions share a label. Proposal results retain the candidate, replay report, timings for feature construction/inference/decoding/verification, and optional score diagnostics. Certification results retain the exact witness, exact cost, elapsed time, and status. The candidate is preserved when a better exact alignment becomes available, allowing direct inspection of the gap.

The ordinary alignment entry point supports fast and certified modes. Separate proposal and certification calls support progressive presentation in the workbench. If proposal construction raises an exception, it returns an explicit failure; the workbench can still request exact computation through its separate certification operation. This distinction matters because the convenience alignment call does not automatically recover every possible software failure. The current “anytime” mode in the Python enumeration uses the exact-comparison path; it is not a learned branch-and-bound algorithm with a stream of improving lower bounds.

For decoding, the runtime precomputes each transition’s preset, postset, label group, and score lookup. Local marking search then operates on sparse token dictionaries and a visited-marking set. These implementation choices reduce overhead without changing the firing semantics. Each proposal still constructs its input features and runtime; the

Table 10. Implementation responsibilities in the `lara_align` package. A module’s output is not given stronger guarantees than the checks it performs.

Module	Responsibility
<code>features.py</code>	Builds typed graph arrays, label IDs, and exact compatibility.
<code>model.py</code>	Encodes inputs and returns learned move scores and auxiliary outputs.
<code>decode.py</code>	Maintains markings, chooses synchronous transitions, and searches for model-move paths.
<code>verify.py</code>	Replays complete move sequences, checks projections, and recomputes costs.
<code>exact.py</code>	Invokes PM4Py’s exact backend and retains concrete transition identities.
<code>certifier.py</code>	Exposes proposal and exact-comparison operations with explicit statuses.
<code>families.py, data.py</code>	Generate behavior families and store inputs, witnesses, and metadata.
<code>training.py</code>	Extracts supervision and computes the full training objective.

reported timing does not assume a globally cached neural representation for a target net. 827
The checkpoint is loaded once before processing a sequence of traces. 828

6.2. Preparing a Log and Reference Model 829

The Setup page accepts XES event logs, including compressed files, and PNML nets. 830
An analyst may provide an independent reference model or discover a model from the 831
uploaded log using Inductive Miner. The latter option is labeled explicitly: checking a 832
model discovered from the same data is a different analysis from checking compliance 833
with an independently specified process. The discovery noise threshold controls whether 834
infrequent behavior is filtered. 835

The page summarizes the log and net, lists activity labels and duplicate-label groups, 836
and groups cases into variants. Grouping is valid for this control-flow setting because 837
cases with the same activity sequence have the same alignment problem against a fixed 838
net and cost policy. It reduces repeated computation while retaining a mapping back to 839
case identifiers. The analyst chooses which variants to process, and the workbench reports 840
the corresponding case coverage. Unprocessed variants are not counted as successful 841
alignments. 842

6.3. Progressive Results and Trace Inspection 843

The Live Run page offers fast candidates, progressive certification, and an exact 844
baseline. In candidate modes it first produces and independently verifies the proposal 845
for a variant. The result can be displayed immediately, while exact work is a subsequent 846
stage when selected. Statuses distinguish a ready candidate, an illegal candidate, exact 847
computation, certification, repair, timeout, and failure. A timeout retains the evidence 848
already obtained; it does not fill in an unknown optimum. 849

The Trace Inspector aligns candidate and exact explanations to common observed 850
event positions. It exposes concrete transition identities, which is especially useful for 851
duplicate labels, and allows the model projection to be replayed through precomputed 852
marking snapshots. An analyst can therefore see both where an event was skipped and 853
which enabled transition the decoder chose. Neural score diagnostics are supplementary 854
preferences. They are not labeled as causal explanations or calibrated certainty about 855
correctness. 856

Results can be exported as variant-level and case-level CSV, detailed JSON, execution logs, and per-alignment move tables. Candidate cost, exact cost, gap, legality, and certification remain separate fields. This makes subsequent analysis auditable: a CSV consumer can select only certified optima or deliberately study the latency and quality of verified candidates. The optional experiment dashboard reads stored metric files for training-curve and benchmark inspection.

6.4. Reproducibility and Experiment Artifacts

The repository includes the reference synthetic corpus, checkpoint, generation configuration, per-epoch metrics, and benchmark outputs. The script `init_data.py` generates the behavior families; `train_model.py` trains and saves checkpoints. Evaluation scripts cover the fixed synthetic test split, scaling inputs, PM4Py approximations, and real logs. The journal source additionally packages the result data used by its plots and records hashes of the source artifacts, so figures can be rebuilt without rerunning training.

Reproducing candidate-quality figures requires the recorded checkpoint and matching inputs. Reproducing timings additionally requires the hardware, software, thread settings, and measurement boundaries in Section 7.1. Existing result files are measurements of their original runs; rebuilding a plot from those files is not a new runtime experiment. The delivery includes commands for both operations and keeps them separate.

The workbench is an implementation artifact, not a user-study result. Its progressive workflow illustrates how to expose the method’s guarantees in practice, but the experiments below measure alignment outputs and algorithmic timings. They do not measure analyst productivity, user satisfaction, or deployment performance under concurrent load.

7. Experimental Evaluation

The evaluation asks whether the candidate is useful before exact repair and whether learning improves that candidate enough to justify its cost. We therefore measure the proposed explanation separately from the exact result. The experiments progress from controlled test families to larger synthetic nets, competing approximations, and real event logs. This order separates the effects of semantic constraints, learned ranking, representation, and input scale.

7.1. Protocol, Metrics, and Comparison Boundaries

The reference experiments use an Intel Core i7-1355U CPU and the approximately 1.7-million-parameter checkpoint selected at epoch 49. Computation is CPU-only, with PyTorch reporting ten intra-operation and ten inter-operation threads; trace calls are sequential. The checkpoint is loaded once before timing. The synthetic test set has 512 rows from 128 generated families, with 64 rows for each motif/representation cell. The scoring parameters stay fixed throughout the test, scaling, and real-log experiments.

The primary metrics are replay acceptance, exact-cost agreement, and the cost gap in Equation (4). We also report full transition-aware move-sequence agreement with the teacher and agreement after replacing transition identities with labels. These sequence measures diagnose imitation of one witness; neither is required for an optimum when several witnesses share its cost. For representation pairs, cost agreement measures whether the same observation receives equal candidate costs on the two equivalent nets.

The guidance ablation preserves the decoder, costs, depth bounds, and test inputs. It disables the neural forward pass and uses deterministic orderings in place of learned rankings. This is a strong baseline because enabledness and bounded model-path search already resolve many cases. It directly answers RQ2; comparison with exact search alone would not isolate the contribution of learning.

For the LARA timing measurements, fast-mode latency includes feature extraction, runtime construction, neural inference when enabled, decoding, and replay. Exact timings include the PM4Py heuristic initialization and search. In the approximation benchmark, the stored timing surrounds the PM4Py alignment call; independent parsing and replay occur afterward. These boundaries are not identical, so small timing differences should not be overinterpreted. All candidate costs are nevertheless compared after replay under the same unit visible-deviation policy. Reported medians describe the recorded runs, not a universal hardware-independent threshold.

The default test decoder uses catch-up depth eight and completion depth 32. Scaling and real-log scripts use completion depth 64 while keeping the catch-up depth at eight. Exact timeouts in the stress benchmark are 30 seconds, and approximation calls receive five seconds. A timeout leaves the exact cost unknown; the corresponding candidate is included in coverage statistics but excluded from cost-gap and optimality calculations that require the optimum. The tables state these denominators explicitly.

For representation-specific guidance differences, 95% percentile intervals resample whole families 10,000 times with seed 13. A cell contains 32 families with two observations each. Resampling families respects the dependence between observations and representations. These intervals describe sampling within the fixed generator and checkpoint; they do not include variability from retraining with different seeds.

7.2. Candidate Quality on the Synthetic Test Split

The first experiment answers whether the fast pipeline produces complete executable explanations and how close they are to the optimum (RQ1). All 512 candidates pass independent replay. Of these, 466 have the exact optimal cost, giving 91.0% optimality. Table 11 reports the remaining diagnostics and Figure 7 shows the full gap distribution.

The mean gap is small because most candidates are optimal and the remaining gaps are between one and four. Sequence agreement is substantially lower than cost agreement. This is expected when several legal explanations are optimal and the teacher stores one of them. The important distinction for the assurance contract is that replay proves feasibility while exact cost comparison proves optimality, regardless of identity agreement.

When certified mode is evaluated on these inputs, it invokes exact search on all 512. The 466 optimal candidates can be confirmed and retained; the remaining 46 are replaced by exact alignments. The 91.0% value therefore counts successful proposals, not traces on which exact search was avoided. This experiment establishes useful candidate quality within the generator, while leaving end-to-end certification acceleration unproven.

7.3. What Does Learned Guidance Add?

The ablation isolates learned ranking from the model semantics. Unguided decoding also returns 512 legal candidates and reaches optimal cost on 445 of them, or 86.9%. Adding

Table 11. Candidate results on the 512-row synthetic test split. Exact costs come from the teacher/reference alignment; candidate metrics precede any exact repair.

Metric	Result
Replay-verified candidates	512/512 (100.0%)
Candidates with optimal cost	466/512 (91.0%)
Full transition-aware move-sequence match	313/512 (61.1%)
Full label-level move-sequence match	320/512 (62.5%)
Mean / median cost gap	0.180 / 0
90th / 95th percentile cost gap	0 / 2
Maximum cost gap	4

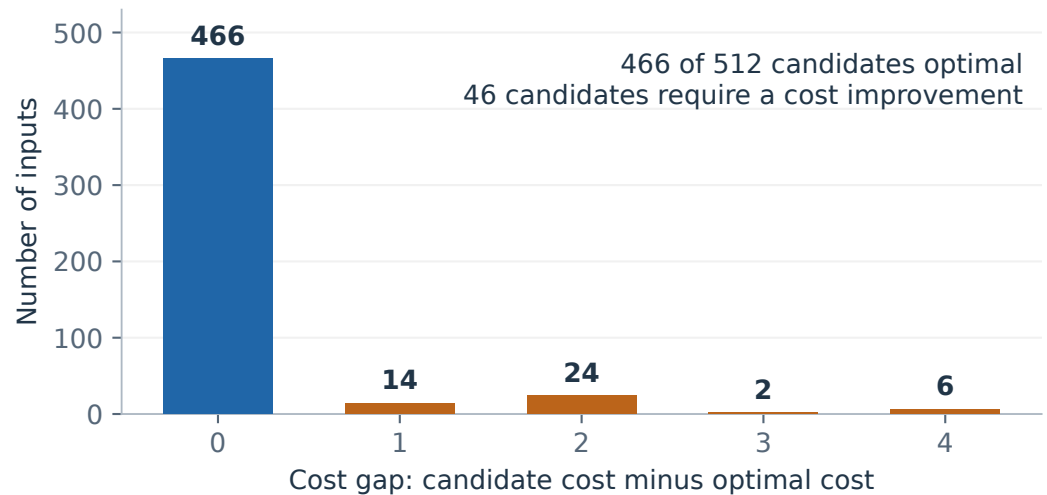


Figure 7. All candidate gaps on the test split. Counts sum to 512; 466 candidates have gap zero and 46 have positive gaps. A low mean gap does not make a particular unchecked candidate optimal.

the learned scorer yields 21 more optimal candidates, an increase of 4.10 percentage points. Mean gap decreases from .258 to .180. Transition-aware sequence agreement rises from 57.8% to 61.1%, and label-level agreement from 58.8% to 62.5%.

These aggregate gains are real for the fixed test set but do not imply that learning is responsible for most successful alignments. The unguided decoder already synchronizes uniquely enabled matches and searches for short model-move paths. Many inputs offer little meaningful ranking choice. To answer where the improvement comes from, Table 12 separates the eight representation cells and Figure 8 shows the corresponding differences.

The duplicate-prefix cell improves from 68.8% to 95.3%, a gain of 26.6 points, with a family-bootstrap interval of [17.2, 35.9] points. This is the case in which the trace suffix can distinguish two currently enabled transitions with the same label. Silent routing already attains 98.4% without learning: its bounded search can postpone the choice until the distinguishing event. Guided and unguided costs coincide on the parallel/interleaving and M cells. RQ2 therefore has a localized answer: the measured gain is strongest where the decoder must commit before structural enabledness resolves the ambiguity.

7.4. Sensitivity to Equivalent Representations

Equivalent models let us examine reuse without changing the visible alignment problem (RQ3). Table 13 compares candidate costs across each representation pair for the same

Table 12. Optimality by motif and representation, with 64 rows per cell. G/U denote guided/unguided decoding; all outputs in both modes pass replay. The mean gap column is for guided candidates.

Motif	Representation	G optimal	U optimal	G mean gap
Ordinary tree	Canonical block	54/64 (84.4%)	55/64 (85.9%)	.344
Ordinary tree	Isomorphic rename	54/64 (84.4%)	49/64 (76.6%)	.344
Duplicate/silent	Duplicate prefix	61/64 (95.3%)	44/64 (68.8%)	.094
Duplicate/silent	Silent routing	63/64 (98.4%)	63/64 (98.4%)	.031
Concurrency	Parallel	58/64 (90.6%)	58/64 (90.6%)	.219
Concurrency	Interleaving	58/64 (90.6%)	58/64 (90.6%)	.219
M pattern	Canonical block	59/64 (92.2%)	59/64 (92.2%)	.094
M pattern	Non-free choice	59/64 (92.2%)	59/64 (92.2%)	.094

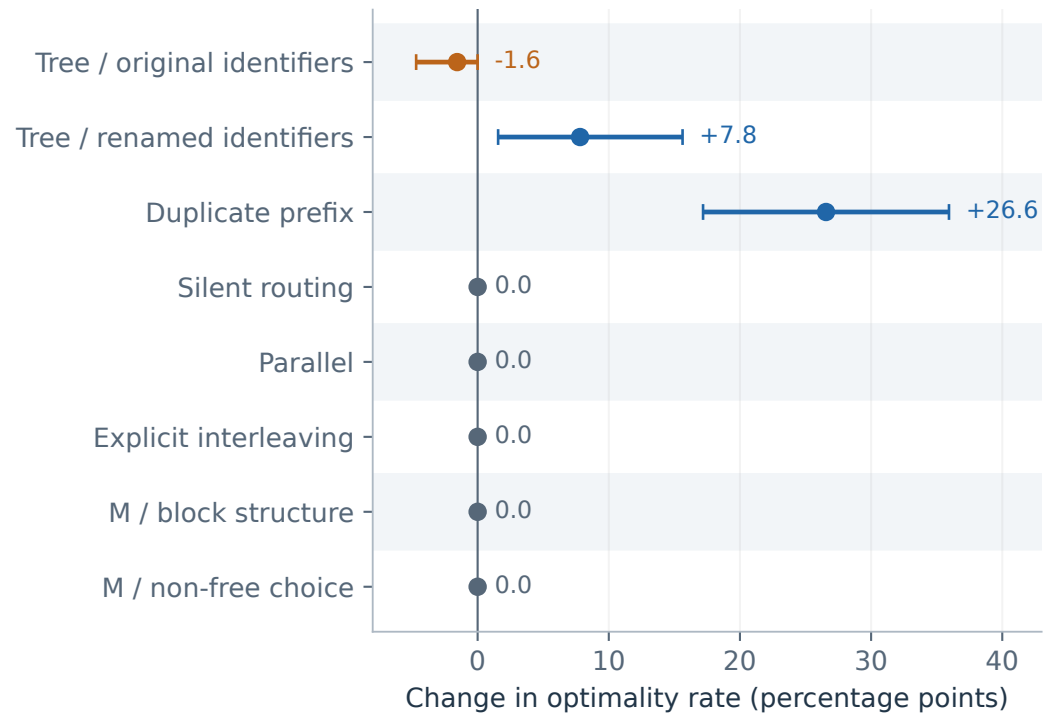


Figure 8. Change in optimality rate when the fixed scorer is enabled. Positive values favor guidance. Intervals are 95% percentile intervals from 10,000 family-level bootstrap resamples; the zero-width intervals indicate identical sampled success differences, not certainty about other generators. The strongest gain is concentrated in duplicate-prefix choices.

observation. The ordinary-tree pair is particularly informative because the two nets differ only in identifiers.

Guided decoding reaches 84.4% optimality on both canonical and renamed trees, with identical paired costs. Unguided decoding falls from 85.9% to 76.6% after renaming. Guidance is thus slightly worse on canonical identifiers, by -1.6 points with interval $[-4.7, 0.0]$, but better on the renamed identifiers by $+7.8$ points with interval $[1.6, 15.6]$. These signs have a common explanation: deterministic name order happens to be favorable on one numbering and less favorable on the other. The learned scorer maintains the same quality across this tested renaming.

Across all representation pairs, agreement rises from 90.2% to 99.2%. This is evidence of stability for the tested transformations, not proof of invariance under every isomorphism or activity relabeling. Ties, numerical behavior, and changed label embeddings can still affect choices. It also does not establish transfer to unseen compilation schemes: all four representation families occur during training.

Table 13. Candidate-cost agreement across paired representations. Each row compares 64 observation pairs. Exact costs agree in every pair. Candidate-cost equality may include two equally suboptimal results.

Equivalent representation pair	Guided agreement	Unguided agreement
Canonical tree / isomorphic rename	100.0%	90.6%
Duplicate prefix / silent routing	96.9%	70.3%
Parallel / explicit interleaving	100.0%	100.0%
Block M / non-free-choice M	100.0%	100.0%
All 256 observation pairs	99.2%	90.2%

7.5. Scaling: Candidate Latency and Quality Together

The scaling experiment tests whether the candidate remains useful when inputs move beyond the small training distribution. A separate generator produces random block-structured nets with sequence, choice, parallelism, loops, duplicate labels, and invisible routing. The nominal sizes are 5, 10, 20, and 40 visible leaf occurrences. An alphabet fraction of .7 permits repeated labels; loop ployout uses redo probability .3 and at most two redos. Deviations are injected at rates .15 and .35, with ten inputs per configuration. These rates are parameters of the stress generator, not the clean/noisy mixture in Section 5.4.

LARA’s guided median ranges from 4.6 to 8.7 ms across this grid. Exact A* is faster on the small configurations. It becomes slower at size 20 with deviation .35, and at size 40 for both rates. At size 40 and deviation .15 the median ratio is approximately $63.9/8.7 = 7.34$ in favor of the fast candidate. One exact call times out at size 40 and deviation .35. These are candidate latency comparisons, not certified-mode speedups.

Quality declines at the same time: at size 40 only 30.0% or 44.4% of comparable candidates are optimal, and mean gaps reach 8.1 or 11.7. Replay remains successful on all 80 candidates, demonstrating executability despite this loss of quality. Unguided decoding takes approximately .2–1.2 ms across the grid and is faster than the exact medians throughout. Guidance contributes little on this stress distribution. The practical question is therefore whether better rankings justify the neural inference cost, not simply whether a neural forward pass is faster than a difficult exact search.

7.6. Comparison with Established Approximations

Competing candidate generators provide a more demanding test than exact search alone. The benchmark uses PM4Py 2.7.23.2 implementations of discounted A* [18], tandem repeats [20], sliding windows [19], fixed-horizon search [17], and subset/edit-distance approximation [22]. Sliding windows use length 20 and at most five candidates; the fixed horizon is four; the configured expansion budget is 100,000 where supported. The subset size is one within each model-variant group and is evaluated only on the held-out corpus. These are concrete settings, not tuned best-case envelopes for each method.

All returned alignments are parsed into transition-aware moves, replayed independently, and rescored with the same costs. This is particularly important for discounted objectives: their internally reported objective need not equal the ordinary visible-deviation count. Table 15 reports the held-out comparison, and Table 16 reports stress results from the approximation benchmark run.

Every tested Petri net search/compression approximation exceeds LARA’s optimality rate on the small corpus. LARA exceeds the particular subset/edit-distance configuration.

Table 14. Scaling results. Size is the requested visible leaf count and $\overline{|T|}$ the rounded mean total transition count. Every configuration has ten legal candidates. n_δ is the number with an exact reference; optimality and mean gap use that denominator. Time columns are median milliseconds per trace; exact timeouts are recorded at 30 seconds.

Size	Deviation	$\overline{ T }$	n_δ	Optimal	Mean gap	Fast ms	Exact ms
5	.15	7	10	90.0%	.2	5.5	1.3
5	.35	6	10	60.0%	1.6	4.6	1.3
10	.15	13	10	80.0%	1.2	5.0	1.8
10	.35	13	10	50.0%	1.7	5.3	2.8
20	.15	28	10	60.0%	3.6	6.9	2.9
20	.35	26	10	60.0%	3.1	6.0	11.7
40	.15	55	10	30.0%	8.1	8.7	63.9
40	.35	54	9	44.4%	11.7	8.6	24.7

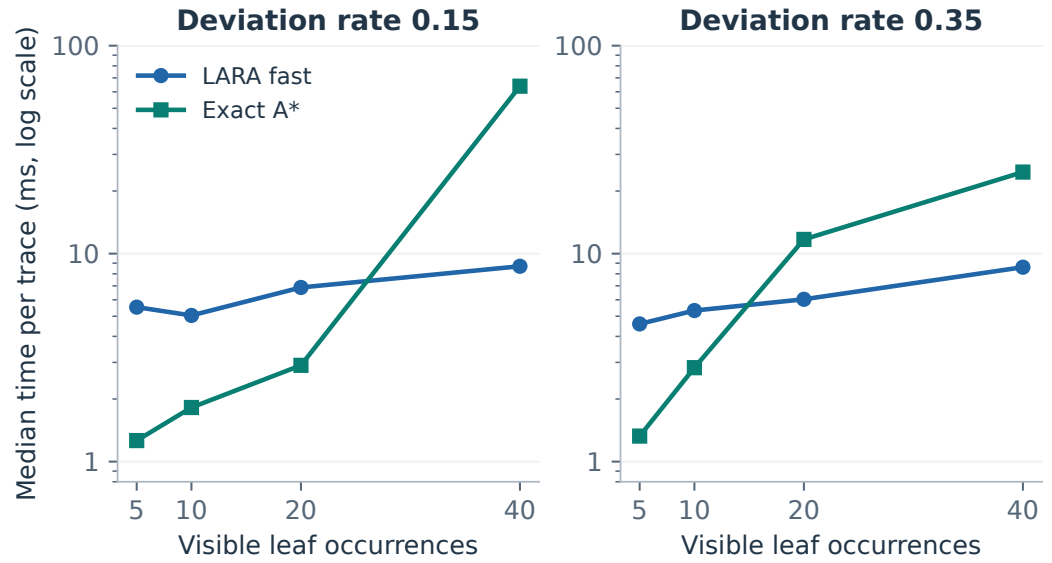


Figure 9. Recorded median latencies on the scaling grid. The vertical axis is logarithmic; the horizontal axis is the number of visible leaf occurrences, not distinct activity labels. Exact search is faster on smaller cells, while its median is larger on several larger cells. These observations do not establish an asymptotic runtime law.

Table 15. Candidate quality on the synthetic test split. All six methods provide 512 legal outputs in this experiment, so the optimality denominator is 512 for every row.

Method	Optimal candidates	Mean gap
LARA guided	466/512 (91.0%)	.180
Discounted A*	490/512 (95.7%)	.090
Tandem repeats	502/512 (98.0%)	.039
Sliding window	512/512 (100.0%)	.000
Fixed horizon	502/512 (98.0%)	.047
Subset/edit distance	360/512 (70.3%)	.637

All held-out traces fit within one sliding window of 20 events, so the perfect result in that row does not exercise the approximation caused by crossing a window boundary. The comparison illustrates why a small benchmark can be useful for diagnosis without being a sufficient stress test.

The stress comparison again favors established approximations in quality. Discounted A* has the same full legal coverage as LARA, a higher exact-cost rate, a smaller mean gap, and a lower recorded median latency. Tandem repeats and sliding windows are even closer to optimal where they return comparable results. Their incomplete coverage must remain visible: a perfect conditional optimum rate is not a result on every attempted input. LARA's full coverage is useful, but it is not unique to the learned method on this grid.

The small and stress comparisons rule out a claim that the present learned greedy decoder is the best general approximation. They instead identify a reusable scoring mechanism with a measured local advantage over its own unguided baseline. Any later claim of competitive overall approximation quality requires a stronger decoder and renewed comparison using common timing boundaries and coverage accounting.

7.7. Application to Public Event Logs without Fine Tuning

Real-log inputs test the use of one checkpoint with different vocabularies, model structures, and trace lengths. We use a building-permit receipt log [32], containing 1434 cases, 116 variants, and 27 activity labels, and the repository's 100-case sample of the

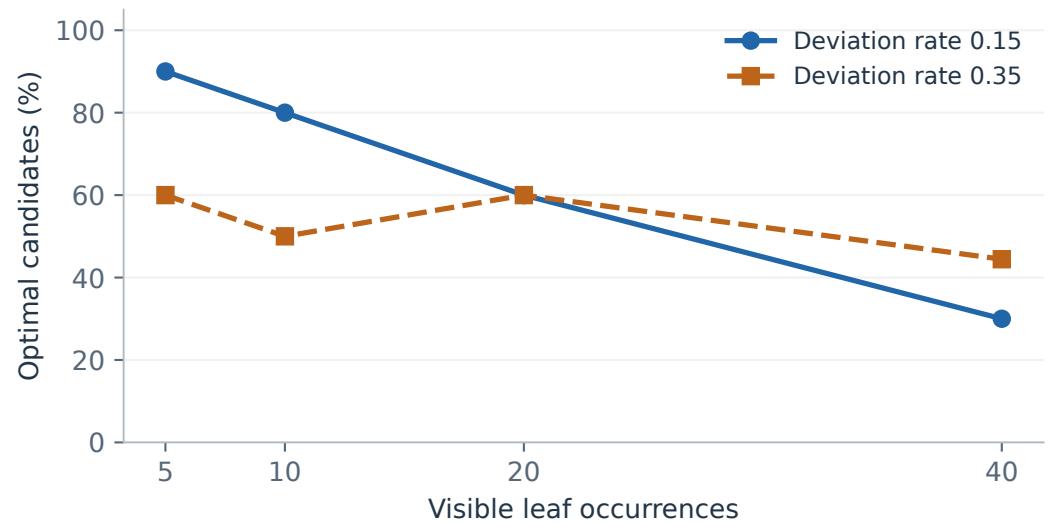


Figure 10. The quality cost of the latency advantage in Figure 9. Optimality declines on larger stress inputs. Each point uses ten exact references except size 40/deviation .35, which uses nine. Small cell sizes and changing random nets explain why the curves need not be monotone.

Table 16. Stress comparison over 80 inputs. Coverage counts legal outputs out of all 80; n_δ counts legal outputs with an exact reference. Optimality and mean gap are conditional on those n_δ inputs. Time is the median over all calls, including unsuccessful calls. Different coverage denominators limit direct quality comparisons.

Method	Coverage	n_δ	Optimal	Mean gap	Median ms
LARA guided	80/80	79	59.5%	3.797	7.69
Discounted A*	80/80	79	75.9%	1.468	1.17
Tandem repeats	78/80	78	93.6%	.128	1.37
Sliding window	79/80	79	100.0%	.000	1.27
Fixed horizon	76/80	76	82.9%	2.263	82.37

road-traffic fines log [33], containing ten variants and ten labels. The road-traffic results refer to that sample, not the complete public log. The longest variants have 25 and nine events, respectively.

For each log, Inductive Miner [34] discovers nets at noise thresholds .0 and .5. Models are discovered from the same log that is subsequently aligned, so this experiment evaluates algorithmic transfer, not independent normative compliance. The positive noise threshold removes infrequent behavior and creates additional deviations against the resulting model. The checkpoint is unchanged; new label strings enter through hashing without any language interpretation or fine tuning.

Table 17. Real-log candidate quality. All variants pass replay. Net size gives places and visible/invisible transitions. Guided and unguided decoding have identical quality values in all four configurations. Mean and maximum gaps are over variants.

Log	Noise	Net size $ P / T_v + T_r $	Optimal (%)		Gap Mean/max
			Variants	Cases	
Road traffic	.0	15/10 + 10	100.0%	100.0%	0/0
Road traffic	.5	13/10 + 7	70.0%	94.0%	.50/3
Receipt	.0	45/27 + 47	50.0%	82.6%	.76/5
Receipt	.5	25/24 + 19	55.2%	74.6%	.82/7

Table 17 distinguishes variants from cases. If variant v occurs f_v times, case-weighted optimality is

$$\frac{\sum_v f_v \mathbf{1}[g_v = 0]}{\sum_v f_v}, \quad (35)$$

whereas variant optimality weights each distinct sequence once. The higher case-weighted values show that frequent variants are easier for this decoder; misses are concentrated in less frequent variants. For the fitting receipt net, all exact costs are zero, but only half of the candidate variants attain zero. A legal, positive-cost candidate on a fitting trace is therefore a concrete example of why replay alone is insufficient to infer nonconformance.

The guided candidate is slower than exact search on the small road-traffic nets and faster on the tested receipt nets (Table 18). Unguided decoding is faster still and has the same candidate quality. These discovered nets have no duplicate visible labels, so they do not exercise the scorer’s strongest demonstrated advantage. Model-move ordering could still matter in principle, but no quality difference is observed here. The experiment supports interface transfer and verified execution under different inputs; it does not demonstrate that the neural scorer improves real-log alignment quality.

7.8. A Failure That Explains the Remaining Gaps

Aggregate rates show how often the decoder fails to be optimal, but a worked failure explains why. Consider test observation $\langle A, D, B, C, C, D \rangle$ in the parallel/interleaving family. Its process permits A, B in either order, then C, D . An optimal alignment skips the early D and one repeated C , giving cost two. LARA instead inserts model moves B, C so that it can synchronize the early D . It reaches the final marking while four observed events remain, forcing four log moves. Its total cost is six, hence gap four.

The error is a decoder commitment, not a replay failure. The decoder tries a bounded model path before considering a log move and never compares that path with skipping the early D . The trained log score is unused, and earlier choices cannot be revised. Better duplicate-label scoring cannot fix every instance of this mechanism. It motivates joint ranking of synchronous, log, and model alternatives and retaining several candidate continuations, as discussed next.

Table 18. Recorded median time per variant in milliseconds. These are candidate/exact timings, not full-log wall-clock speedups. Including the unguided decoder makes clear that the real-log candidate advantage is not evidence of a learned-quality benefit.

Log	Noise	Guided fast	Unguided fast	Exact A*
Road traffic	.0	5.39	.56	1.92
Road traffic	.5	5.22	.48	1.56
Receipt	.0	16.05	6.01	31.04
Receipt	.5	6.65	.61	12.72

A. Skip the early D and one repeated C

log	A	D	B	C	C	D	optimal cost 2
model	A	»	B	C	»	D	

4 synchronous moves + 2 log moves

B. Catch up to the early D , then finish too soon

log	A	»	»	D	B	C	C	D
model	A	B	C	D	»	»	»	»

2 model moves 4 log moves after completion

Candidate cost $2 + 4 = 6$; optimal cost 2; gap 4.
Every firing is legal, but the early decision harms the suffix.

Figure 11. A legal but suboptimal explanation of $\langle A, D, B, C, C, D \rangle$. Each visible deviation is amber. Invisible routing is omitted, and model rows display activity labels for readability. The upper explanation costs two; the lower costs six. The early attempt to synchronize D causes both the two inserted model activities and the four subsequent skipped events.

8. Discussion and Limitations

The results are most useful when they guide a concrete choice about when to generate a candidate and what to infer from it. This section connects the three research questions to that choice, distinguishes the semantic contract from empirical performance, and identifies the changes needed for stronger claims. It also records limits of the evidence so that the observed transfer is not mistaken for general-purpose alignment competence.

8.1. Interpreting the Three Research Questions

For RQ1, the experiments show that the decoder and replay pipeline produced legal candidates on all evaluated inputs, with 91.0% optimality on the controlled test split. The formal guarantee is conditional: when replay accepts, the candidate is a complete executable explanation and its cost is an upper bound. The observed 100% acceptance rate does not prove that the bounded decoder always reaches the final marking. Larger gaps on stress inputs and the worked failure show that a reliable legality check can coexist with a weak candidate policy.

For RQ2, learning has a measurable but concentrated effect. The duplicate-prefix improvement demonstrates that global trace context can help decide among equal labels before the decoder reaches their distinguishing continuation. The identifier-renaming experiment shows more stable candidate costs than the name-ordered baseline. The remaining motifs show no cost improvement. The evidence therefore supports using learned rankings for selected ambiguities, rather than attributing the overall success rate to the network.

For RQ3, one checkpoint accepts new net sizes and label vocabularies without fine tuning. The candidate can arrive faster than the tested exact backend on larger receipt models and selected stress cells. However, guided inference imposes a substantial floor on small cases, quality degrades on larger stress inputs, and established approximations perform better overall. The real-log quality is identical without learning. Reuse is demonstrated at the interface and candidate-pipeline level, while broad learned generalization remains limited.

8.2. What a Verified Candidate Can and Cannot Tell an Analyst

A verified candidate is useful because it is an executable explanation of the observed case. It can support visualization, an initial inspection of deviations, and a bound on the best achievable cost. Its deviation locations should nevertheless be read in the light of its optimization status. The fitting receipt model makes this concrete: a positive candidate cost does not prove that the trace deviates from the model, because a zero-cost alignment may exist. If the goal is to assert the minimum deviation count, exact evidence remains necessary.

Likewise, equal optimal costs do not identify a unique diagnostic story. Different optimal alignments may allocate deviations or synchronize independent activities differently. Certification establishes minimal cost under the chosen policy, not the uniquely correct account of what happened in the organization. Business interpretation still depends on the model, event-data quality, and the suitability of the cost function.

Progressive presentation is a practical application of these distinctions. A workbench can show a candidate and its replay status immediately, then add exact comparison when available. The current implementation supports that sequence. A deployment policy that starts exact search first and invokes candidate generation only for slow cases could reduce unnecessary inference, but this policy has not been evaluated. The threshold would depend on model structure, hardware, expected quality, and the analyst's need for a certificate.

8.3. Threats to Validity

Training distribution. The corpus is small and intentionally structured. It contains four motif types, three of which use fixed controlled constructions. Family-level splitting prevents separation of a generated family's views, but it does not exclude repeated visible languages or motifs across splits. Holding out whole motif classes, unseen compilation schemes, or larger and more varied net collections would provide stronger evidence of structural transfer. Training on abstract labels also leaves business semantics, resources, and time outside the study.

Teacher and objective. Exact targets are verified for the supplied cost model, but only one optimal witness is retained. Sequence agreement therefore mixes candidate quality with witness choice. The masking rule for duplicate-prefix targets is motif-specific. Moreover, the loss supervises a log head that decoding ignores and auxiliary region heads whose contribution has not been isolated. These choices make the trained model a useful experimental prototype, but they do not establish that the objective is the best one for candidate cost.

Statistical coverage. The main result uses one checkpoint and one training seed. Bootstrap intervals resample families for that fixed model; they are not intervals over retraining or target domains. Stress cells contain only ten inputs, and the real-log study uses two logs with one restricted road-traffic sample. Observed perfect coverage or agreement in these collections should not be generalized to arbitrary systems.

Comparison protocol. The evaluation uses one exact backend and concrete approximation configurations. Alternatives such as stronger exact implementations, other window sizes, or larger representative subsets may change the comparison. Stored PM4Py approximation times exclude the independent wrapper replay, whereas LARA proposal timing includes it. Hardware scheduling, warm-up effects, and Python overhead can affect millisecond-scale differences. The strongest present comparative evidence concerns the replayed costs and explicit coverage; fine-grained timing claims require a common harness and repeated measurements.

Real-log interpretation. Models are discovered from the same logs used for alignment. This is appropriate for testing the algorithm on different structures and labels, but it does

not establish compliance with an independent organizational specification. Variant and case weighting answer different questions, and selecting a limited set of variants in an interactive run changes the covered population. The paper therefore reports both quality weightings and states the scope of the road-traffic sample.

Semantic and computational scope. The evaluated setting uses ordinary control-flow nets and fixed unit visible-deviation costs. Neural features omit explicit arc weights, current decoder markings, and data guards. The scorer uses dense attention, and the decoder’s depth bounds do not bound the width of marking exploration. Neither component has a demonstrated constant-latency guarantee. The guarantee that survives these implementation limits is the conditional replay/certification contract.

8.4. Methodological Extensions Suggested by the Evidence

The failure in Figure 11 suggests comparing a log move with model catch-up before committing. A decoder could rank synchronous, log, and model alternatives jointly and retain a small beam of partial alignments. Recomputing scores from the current marking or predicting model preferences per event position could help it respond to earlier choices. These changes would increase inference or search cost, so they should be assessed against both the current unguided decoder and strong non-neural approximations.

A separate extension concerns certification. A replay-verified candidate supplies a feasible incumbent cost. An exact search algorithm can potentially use that cost for pruning while relying on admissible lower bounds for correctness. Such an integration would need to expose the relevant solver interface, preserve exactness, and demonstrate reduced total work. Merely comparing a candidate with an independently computed optimum, as done here, cannot support a certification-speedup claim.

Training could also focus on the decisions that the active decoder actually makes, collect multiple optimal witnesses, or penalize downstream candidate gaps more directly. Additional structural diversity and deliberate tests of hash relabeling would strengthen evidence for reuse. The auxiliary regions would require independent validation before being treated as meaningful process decompositions. These are research directions motivated by observed limitations, not capabilities of the evaluated checkpoint.

9. Conclusions

The aim of this study is to determine a useful role for reusable learning in alignment-based conformance checking while keeping the meaning of an explanation explicit. LARA combines a pretrained graph-and-trace scorer with a marking-aware decoder, independent replay, and optional exact comparison. The scorer proposes move preferences; replay establishes executability and an upper bound; successful exact computation establishes the optimum or supplies a replacement.

On the controlled synthetic test set, guidance increases optimal candidates from 445 to 466 out of 512, with its largest contribution on duplicate-prefix choices. All evaluated candidates pass replay, but candidate quality falls on larger stress inputs. Real-log experiments show that the fixed checkpoint can be used with new vocabularies and model structures, while providing no quality improvement over the unguided decoder on those logs. Established approximations generally perform better in the direct comparisons, and the current certified mode still runs exact search for every trace.

The resulting contribution is a transparent, reproducible study of learned candidate scoring within an alignment pipeline whose guarantees come from process semantics. The most promising next steps are stronger decisions among competing move types, broader tests of structural transfer, and using verified incumbents to reduce exact search work. Their value must be demonstrated in candidate quality and total computation, with the same explicit separation between a plausible prediction and a proven alignment property. **Data**

Availability Statement: The implementation, synthetic data, pretrained checkpoint, and experiment scripts are available in the LARA repository at <https://github.com/fit-alessandro-berti/lara-align>. The journal source includes the numerical inputs used to draw its result figures. The public event-log sources are cited in Section 7.7.

Abbreviations

The following abbreviations are used in this manuscript:

A*	Heuristic shortest-path search
BFS	Breadth-first search
CPU	Central processing unit
GNN	Graph neural network
LARA	Learned Alignment with Replay Assurance
PNML	Petri Net Markup Language
RQ	Research question
XES	eXtensible Event Stream

References

1. van der Aalst, W.M.P. *Process Mining: Data Science in Action, Second Edition*; Springer: Berlin, Heidelberg, 2016.
2. Adriansyah, A.; van Dongen, B.F.; van der Aalst, W.M.P. Conformance Checking Using Cost-Based Fitness Analysis. In Proceedings of the EDOC, Los Alamitos, CA, USA, 2011; pp. 55–64.
3. Carmona, J.; van Dongen, B.; Solti, A.; Weidlich, M. *Conformance Checking: Relating Processes and Models*; Springer: Cham, Switzerland, 2018. <https://doi.org/10.1007/978-3-319-99414-7>.
4. Bommasani, R.; Hudson, D.A.; Adeli, E.; Altman, R.; Arora, S.; et al. On the Opportunities and Risks of Foundation Models. *arXiv* **2021**, *arXiv:2108.07258*. <https://doi.org/10.48550/arXiv.2108.07258>.
5. Rizk, Y.; Venkateswaran, P.; Isahagian, V.; Muthusamy, V. A Case for Business Process-Specific Foundation Models. *arXiv* **2022**, *arXiv:2210.14739*. <https://doi.org/10.48550/arXiv.2210.14739>.
6. van Zelst, S.J.; Bolt, A.; van Dongen, B.F. Computing Alignments of Event Data and Process Models. *Trans. Petri Nets Other Model. Concurr.* **2018**, *13*, 1–26.
7. van Dongen, B.F. Efficiently Computing Alignments - Using the Extended Marking Equation. In Proceedings of the BPM, Cham, Switzerland, 2018; Vol. 11080, *Lecture Notes in Computer Science*, pp. 197–214.
8. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention is All you Need. In Proceedings of the NIPS, Red Hook, NY, USA, 2017; pp. 5998–6008.
9. Gilmer, J.; Schoenholz, S.S.; Riley, P.F.; Vinyals, O.; Dahl, G.E. Neural Message Passing for Quantum Chemistry. In Proceedings of the Proceedings of the 34th International Conference on Machine Learning. PMLR, 2017, Vol. 70, *Proceedings of Machine Learning Research*, pp. 1263–1272.
10. Rozinat, A.; van der Aalst, W.M.P. Conformance Checking of Processes Based on Monitoring Real Behavior. *Information Systems* **2008**, *33*, 64–95. <https://doi.org/10.1016/j.is.2007.07.001>.
11. Berti, A.; van der Aalst, W.M.P. A Novel Token-Based Replay Technique to Speed Up Conformance Checking and Process Enhancement. In *Transactions on Petri Nets and Other Models of Concurrency XV*; Springer: Berlin, Heidelberg, 2021; Vol. 12530, *Lecture Notes in Computer Science*, pp. 1–26. https://doi.org/10.1007/978-3-662-63079-2_1.
12. Casas-Ramos, J.; Mucientes, M.; Lama, M. REACH: Researching Efficient Alignment-Based Conformance Checking. *Expert Systems with Applications* **2024**, *241*, 122467. <https://doi.org/10.1016/j.eswa.2023.122467>.
13. Schwanen, C.T.; Pakusa, W.; van der Aalst, W.M.P. Computational Complexity of Alignments. *CoRR* **2026**, *abs/2603.05331*.
14. van der Aalst, W.M.P. Decomposing Petri nets for process mining: A generic approach. *Distributed and Parallel Databases* **2013**, *31*, 471–507. <https://doi.org/10.1007/s10619-013-7127-5>.

15. Muñoz-Gama, J.; Carmona, J.; van der Aalst, W.M.P. Single-Entry Single-Exit Decomposed Conformance Checking. *Information Systems* **2014**, *46*, 102–122. <https://doi.org/10.1016/j.is.2014.04.003>.
16. Lee, W.L.J.; Verbeek, H.M.W.; Munoz-Gama, J.; van der Aalst, W.M.P.; Sepúlveda, M. Recomposing conformance: Closing the circle on decomposed alignment-based conformance checking in process mining. *Inf. Sci.* **2018**, *466*, 55–91.
17. van Dongen, B.F.; Carmona, J.; Chatain, T.; Taymouri, F. Aligning Modeled and Observed Behavior: A Compromise Between Computation Complexity and Quality. In Proceedings of the CAiSE, Cham, Switzerland, 2017; Vol. 10253, *Lecture Notes in Computer Science*, pp. 94–109. https://doi.org/10.1007/978-3-319-59536-8_7.
18. Boltenhagen, M.; Chatain, T.; Carmona, J. A Discounted Cost Function for Fast Alignments of Business Processes. In Proceedings of the BPM, Cham, Switzerland, 2021; Vol. 12875, *Lecture Notes in Computer Science*, pp. 252–269. https://doi.org/10.1007/978-3-030-85469-0_17.
19. Bogdanov, E.; Cohen, I.; Gal, A. A Scalable and Near-Optimal Conformance Checking Approach for Long Traces. *CoRR* **2024**, *abs/2406.05439*.
20. Reißner, D.; Armas-Cervantes, A.; Rosa, M.L. Efficient Conformance Checking Using Approximate Alignment Computation with Tandem Repeats. *CoRR* **2020**, *abs/2004.01781*.
21. Schuster, D.; van Zelst, S.J.; van der Aalst, W.M.P. Alignment Approximation for Process Trees. In *Process Mining Workshops*; Springer: Cham, Switzerland, 2021; Vol. 406, *Lecture Notes in Business Information Processing*, pp. 247–259. https://doi.org/10.1007/978-3-030-72693-5_19.
22. Sani, M.F.; van Zelst, S.J.; van der Aalst, W.M.P. Conformance Checking Approximation Using Subset Selection and Edit Distance. In Proceedings of the CAiSE, Cham, Switzerland, 2020; Vol. 12127, *Lecture Notes in Computer Science*, pp. 234–251. https://doi.org/10.1007/978-3-030-49435-3_15.
23. Padró, L.; Carmona, J. Computation of Alignments of Business Processes through Relaxation Labeling and Local Optimal Search. *Information Systems* **2022**, *104*, 101703. <https://doi.org/10.1016/j.is.2020.101703>.
24. Tax, N.; Verenich, I.; Rosa, M.L.; Dumas, M. Predictive Business Process Monitoring with LSTM Neural Networks. In *Advanced Information Systems Engineering*; Springer: Cham, Switzerland, 2017; Vol. 10253, *Lecture Notes in Computer Science*, pp. 477–492. https://doi.org/10.1007/978-3-319-59536-8_30.
25. Bukhsh, Z.A.; Saeed, A.; Dijkman, R.M. ProcessTransformer: Predictive Business Process Monitoring with Transformer Network. *CoRR* **2021**, *abs/2104.00721*.
26. Sommers, D.; Menkovski, V.; Fahland, D. Process Discovery Using Graph Neural Networks. In Proceedings of the Proceedings of the International Conference on Process Mining. IEEE, 2021, pp. 40–47. <https://doi.org/10.1109/ICPM53251.2021.9576849>.
27. Genga, L.; Winter, K. Artificial intelligence in conformance checking: state of the art and research agenda. *Process Sci.* **2025**, *2*. <https://doi.org/10.1007/s44311-025-00015-7>.
28. Bengio, Y.; Lodi, A.; Prouvost, A. Machine Learning for Combinatorial Optimization: A Methodological Tour d’Horizon. *European Journal of Operational Research* **2021**, *290*, 405–421. <https://doi.org/10.1016/j.ejor.2020.07.063>.
29. Yonetani, R.; Taniai, T.; Barekatin, M.; Nishimura, M.; Kanezaki, A. Path Planning Using Neural A* Search. In Proceedings of the Proceedings of the 38th International Conference on Machine Learning. PMLR, 2021, Vol. 139, *Proceedings of Machine Learning Research*, pp. 12029–12039.
30. Loshchilov, I.; Hutter, F. Decoupled Weight Decay Regularization. In Proceedings of the International Conference on Learning Representations, 2019.
31. Berti, A.; van Zelst, S.J.; Schuster, D. PM4Py: A process mining library for Python. *Softw. Impacts* **2023**, *17*, 100556.
32. Buijs, J.C.A.M. Receipt phase of an environmental permit application process (WABO), CoSeLoG project, 2014. <https://doi.org/10.4121/uuid:a07386a5-7be3-4367-9535-70bc9e77dbe6>.
33. de Leoni, M.; Mannhardt, F. Road Traffic Fine Management Process, 2015. <https://doi.org/10.4121/uuid:270fd440-1057-4fb9-89a9-b699b47990f5>.
34. Leemans, S.J.J.; Fahland, D.; van der Aalst, W.M.P. Discovering Block-Structured Process Models from Event Logs - A Constructive Approach. In Proceedings of the Petri Nets, Berlin, Heidelberg, 2013; Vol. 7927, *Lecture Notes in Computer Science*, pp. 311–329.